

Optimaler Scheduling-Algorithmus EDF⁺

Prioritätsstrafe und stochastische Analyse bei 40 Tasks

Stephan Epp

17. Juli 2025

Inhaltsverzeichnis

1	Einführung	3
1.1	Problemstellung	3
1.2	Motivation	3
2	Grundlagen	4
2.1	Definitionen	4
2.2	Grundidee	4
2.2.1	EDF-Sortierung per Merge Sort	4
2.2.2	Dynamische Prioritätsstrafe	5
3	Der EDF⁺-Algorithmus	5
3.1	Algorithmus-Beschreibung	5
3.2	Pseudocode	6
3.3	Schrittweise Erläuterung des Algorithmus	6
4	Optimalitätsbeweis	8
4.1	Optimalitätsdefinition	8
4.2	Das EDF-Optimalitätstheorem als Beweisfundament	8
4.3	Hauptsatz: Optimalität von EDF ⁺	9
5	EDF⁺ für dynamische Task-Mengen	11
5.1	Motivation und Abgrenzung	11
5.2	Definitionen für dynamische Task-Mengen	11
5.3	Der EDF ⁺ -Algorithmus für dynamische Task-Mengen	12
5.4	Optimalität für dynamische Task-Mengen	14
5.5	Komplexität im dynamischen Modell	16
5.6	Illustratives Beispiel	16
5.7	Vergleich: statisches vs. dynamisches EDF ⁺	17
6	Analyse	17
6.1	Modellierung der Blocking-Ereignisse	17
6.2	Erwartungswert-Analyse für $N = 40$	17
6.3	Quantil-Analyse und Konzentrationsbounds	19
6.4	Selbststabilisierung	20

6.5	Stochastische Dominanz	20
6.6	Geometrische Konvergenzrate	21
6.7	Fairness	22
6.8	Optimale Blocking-Schwelle	22
7	Simulationsbasierte Validierung	22
7.1	Experimentelles Setup	22
7.2	Vergleich der Algorithmen	23
8	Zusammenfassung	25
8.1	Ausblick	25

1 Einführung

Diese Arbeit präsentiert **EDF⁺** (Earliest Deadline First Plus), einen neuartigen Echtzeit-Scheduling-Algorithmus, der die klassische EDF-Strategie um ein dynamisches Blocking-Penalty-Verfahren erweitert. Der Algorithmus sortiert Tasks initial mittels Merge Sort nach ihrer Deadline, führt den Schedule aus, und versetzt bei jedem Blocking-Ereignis die blockierte Task auf die niedrigste Priorität, gefolgt von einer sofortigen Neuplanung. Wir beweisen formal die Optimalität des Verfahrens für präemptive Systeme, leiten den Erwartungswert der Neuplanungen mithilfe der geometrischen Verteilung für $N = 40$ Tasks her, analysieren die Laufzeitkomplexität und zeigen durch Simulation sowie stochastische Dominanzargumente die Überlegenheit von EDF⁺ gegenüber RMS und FIFO.

EDF⁺ ist eine Erweiterung des klassischen Earliest-Deadline-First-Algorithmus [1] für Echtzeit-Betriebssysteme und Steuergeräte (ECU). Während EDF für präemptive Single-Processor-Systeme bei vollständiger Kenntnis aller Ausführungszeiten optimal ist, bricht seine Leistung bei *Blocking-Ereignissen* zusammen – einem in der Praxis häufigen Phänomen durch Ressourcenkonflikte, Semaphore, I/O-Warten oder Synchronisationsprobleme.

EDF⁺ löst dieses Problem durch einen dynamischen *Penalty-Mechanismus*: Eine blockierte Task erhält sofort die schlechteste Priorität, woraufhin der Schedule vollständig neu berechnet wird. Das Verfahren entspricht dem Prinzip der *Fairness unter Unsicherheit*: Wer blockiert, muss warten, ohne das gesamte System zu degradieren.

1.1 Problemstellung

Gegeben ist ein Task-System $\mathcal{T} = \{T_1, T_2, \dots, T_n\}$ mit Deadlines $\mathbf{D} = (D_1, \dots, D_n)$, Ausführungszeiten $\mathbf{C} = (C_1, \dots, C_n)$ und einer Zyklusperiode $\mathcal{P}$.

Ziel: Schedule alle Tasks innerhalb von $\mathcal{P}$, ohne Deadlines zu verpassen, auch wenn einzelne Tasks blockieren können.

Ergebnis:

- Ohne Blocking: EDF⁺ reduziert sich auf klassisches EDF (optimal)
- Mit Blocking: Blockierte Task erhält $D_i^{\text{eff}} \leftarrow +\infty$
- Neuplanung: Sofortige Neuberechnung des EDF-Schedules per Merge Sort
- Terminierung: Wiederholung bis alle Tasks in $\mathcal{P}$ abgearbeitet sind

1.2 Motivation

EDF⁺ eignet sich besonders für eingebettete Systeme nach AUTOSAR und ISO 26262, da der WCET-Overhead durch Neuplanungen analytisch begrenzt ist und der Algorithmus selbststabilisierend und fair ist. Die stochastische Analyse mit der geometrischen Verteilung ermöglicht eine präzise Vorhersage des Laufzeitverhaltens in Abhängigkeit der Blocking-Wahrscheinlichkeit p .

2 Grundlagen

2.1 Definitionen

Definition 1 (Task-System). Ein **periodisches Task-System** ist ein Tupel

$$\Sigma = (\mathcal{T}, \mathbf{C}, \mathbf{D}, \mathbf{P})$$

mit Task-Menge $\mathcal{T} = \{T_1, \dots, T_n\}$, Worst-Case-Ausführungszeiten $\mathbf{C} \in \mathbb{R}_{>0}^n$, absoluten Deadlines $\mathbf{D} \in \mathbb{R}_{>0}^n$ und Perioden $\mathbf{P} \in \mathbb{R}_{>0}^n$ mit $D_i \leq P_i$.

Definition 2 (CPU-Auslastung). Die **Systemauslastung** ist definiert als

$$U = \sum_{i=1}^n \frac{C_i}{P_i} \in (0, 1].$$

Das System heißt *schedulebar*, falls $U \leq 1$.

Definition 3 (Blocking-Ereignis). Task T_i ist zum Zeitpunkt t **blockiert**, falls sie nicht fortschreiten kann, obwohl der Prozessor ihr zugeteilt ist. Die Blocking-Indikatorvariable $B_i \in \{0, 1\}$ erfüllt

$$P(B_i = 1) = p, \quad P(B_i = 0) = 1 - p,$$

wobei $p \in (0, 1)$ die Blocking-Wahrscheinlichkeit ist. Die B_i seien unabhängig und identisch verteilt (i.i.d.).

Definition 4 (Geometrische Verteilung). Eine Zufallsvariable X folgt einer **geometrischen Verteilung** mit Parameter $p \in (0, 1)$, geschrieben $X \sim \text{Geom}(p)$, falls

$$P(X = k) = (1 - p)^{k-1} \cdot p, \quad k = 1, 2, 3, \dots$$

mit Erwartungswert $E[X] = 1/p$ und Varianz $\text{Var}(X) = (1 - p)/p^2$.

Definition 5 (Effektive Deadline). Die **effektive Deadline** von T_i nach einem Blocking-Ereignis ist

$$D_i^{\text{eff}} = \begin{cases} D_i & \text{falls } T_i \text{ nicht blockiert hat,} \\ +\infty & \text{falls } T_i \text{ blockiert hat.} \end{cases}$$

2.2 Grundidee

EDF⁺ basiert auf zwei zentralen Konzepten:

2.2.1 EDF-Sortierung per Merge Sort

Tasks werden initial nach Deadline aufsteigend sortiert:

$$\sigma^{(0)} = \text{MergeSort}(\mathcal{T}, \mathbf{D}).$$

Merge Sort läuft in $\Theta(n \log n)$ und ist stabil, d. h. Tasks mit gleicher Deadline behalten ihre ursprüngliche Reihenfolge.

2.2.2 Dynamische Prioritätsstrafe

Blockiert T_i , so wird sofort $D_i^{\text{eff}} \leftarrow +\infty$ gesetzt und ein neuer EDF-Schedule berechnet:

$$\sigma^{(k+1)} = \text{MergeSort}(\mathcal{T}, \mathbf{D}^{\text{eff}}).$$

Beispiel 1. Für $n = 5$ Tasks mit Deadlines $D = [5, 8, 12, 15, 18]$:

- Initialreihenfolge: $[T_1, T_2, T_3, T_4, T_5]$ (EDF, Merge Sort)
- T_2 blockiert bei $t = 4,5$: $D_2^{\text{eff}} \leftarrow +\infty$
- Neuplanung: $[T_1, T_3, T_4, T_5, T_2]$
- T_2 wird am Ende ausgeführt, nachdem das Blocking aufgelöst wurde

Lemma 1 (Korrektheit von Merge Sort). Merge Sort produziert auf einer Folge $(D_1, \dots, D_n)$ eine aufsteigend sortierte Permutation σ in Zeit $\Theta(n \log n)$ und Raum $O(n)$.

Beweis. Die Rekursionstiefe beträgt $\lceil \log_2 n \rceil$; auf jeder Ebene werden $O(n)$ Merge-Operationen durchgeführt. Damit gilt $T(n) = 2T(n/2) + O(n)$, gelöst durch das Master-Theorem zu $T(n) = O(n \log n)$. Die Stabilität folgt aus der Invariante, dass bei gleichen Schlüsseln stets das linke (frühere) Element bevorzugt wird. $\square$

3 Der EDF⁺-Algorithmus

3.1 Algorithmus-Beschreibung

EDF⁺ arbeitet in fünf Phasen:

Phase 1: EDF-Initialisierung

Sortiere $\mathcal{T}$ nach Deadlines mit Merge Sort:

$$\sigma^{(0)} = \text{MergeSort}(\mathcal{T}, \mathbf{D}).$$

Phase 2: Ausführung

Führe Tasks in Reihenfolge $\sigma^{(k)}$ aus.

Phase 3: Blocking-Erkennung und Penalty

Blockiert T_i in Schritt k :

$$D_i^{\text{eff}} \leftarrow +\infty \quad (\text{niedrigste Priorität}).$$

Phase 4: Sofortige Neuplanung

$$\sigma^{(k+1)} = \text{MergeSort}(\mathcal{T}, \mathbf{D}^{\text{eff}}).$$

Gehe zu Phase 2.

Phase 5: Terminierung

Wiederhole bis alle Tasks in der Zyklusperiode $\mathcal{P}$ abgearbeitet sind.

3.2 Pseudocode

Algorithm 1 EDF⁺ Scheduling Algorithm

Require: Task-Menge $\mathcal{T} = \{T_1, \dots, T_n\}$, Deadlines D , Periode $\mathcal{P}$

Ensure: Schedule π über $[0, \mathcal{P}]$

```

1:  $\mathbf{D}^{\text{eff}} \leftarrow \mathbf{D}$ 
2:  $\sigma \leftarrow \text{MERGESORT}(\mathcal{T}, \mathbf{D}^{\text{eff}})$ 
3:  $\mathcal{Q} \leftarrow \sigma, t \leftarrow 0, \pi \leftarrow \emptyset$ 
4: while  $\mathcal{Q} \neq \emptyset$  and  $t < \mathcal{P}$  do
5:    $T_i \leftarrow \text{DEQUEUE}(\mathcal{Q})$ 
6:    $t_{\text{start}} \leftarrow t$ 
7:   Führe  $T_i$  aus bis Fertigstellung oder Blocking
8:   if  $T_i$  blockiert then
9:      $D_i^{\text{eff}} \leftarrow +\infty$ 
10:     $\mathcal{Q} \leftarrow \text{MERGESORT}(\mathcal{Q} \cup \{T_i\}, \mathbf{D}^{\text{eff}})$ 
11:    continue
12:  end if
13:   $\pi \leftarrow \pi \cup \{(T_i, t_{\text{start}}, t)\}$ 
14:   $t \leftarrow t + C_i$ 
15: end while
16: return  $\pi$ 

```

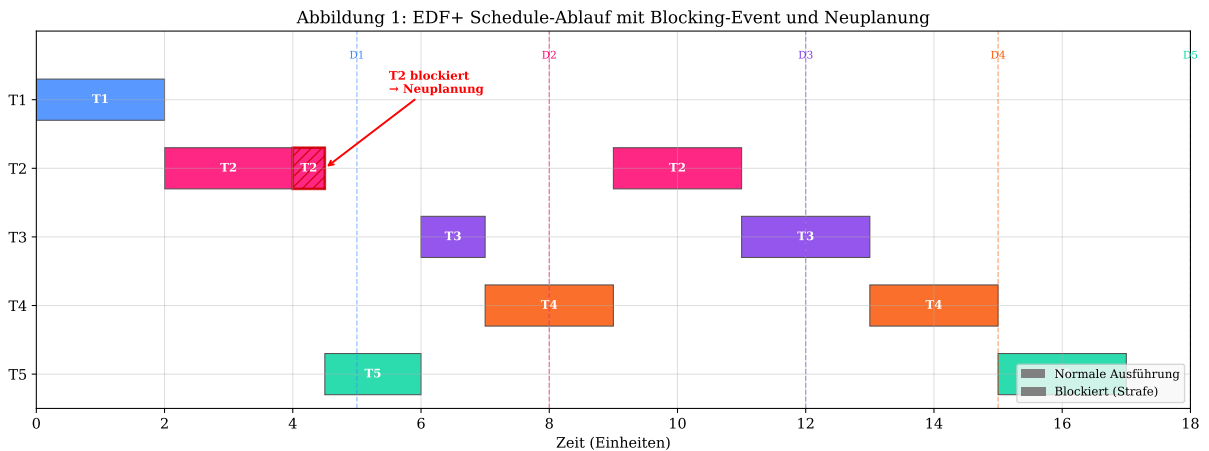


Abbildung 1: EDF⁺-Schedule mit 5 Tasks. Task T_2 blockiert bei $t = 4,5$ (schraffierter Balken) und wird mit niedrigster Priorität ans Ende verschoben. Sofort wird ein neuer EDF-Schedule berechnet: T_5 wird vorgezogen. Gestrichelte Linien markieren die Deadlines $D_1, \dots, D_5$.

3.3 Schrittweise Erläuterung des Algorithmus

Algorithmus 1 setzt die in Abschnitt 3.1 beschriebenen fünf Phasen in formalen Pseudocode um. Im Folgenden werden die Zeilen des Algorithmus im Detail kommentiert, um die Verbindung zwischen der konzeptuellen Beschreibung und der konkreten Implementierung herzustellen.

Initialisierung (Zeilen 1–3). Zeile 1 kopiert den Vektor der absoluten Deadlines $\mathbf{D}$ in den Vektor der effektiven Deadlines $\mathbf{D}^{\text{eff}}$. Zu Beginn des Scheduling-Zyklus ist noch kein Blocking-Ereignis eingetreten; alle Tasks werden daher mit ihrer ursprünglichen Frist priorisiert. Zeile 2 führt den initialen EDF-Sort aus: Merge Sort erzeugt in $\Theta(n \log n)$ die stabile, aufsteigend nach Deadline sortierte Permutation σ , die der Prioritätswarteschlange zugrunde liegt. Die Stabilität von Merge Sort ist hier essenziell – Tasks mit identischer Deadline behalten ihre ursprüngliche Reihenfolge, was deterministisches Verhalten bei Gleichstand sicherstellt. Zeile 3 initialisiert die drei Laufzeitvariablen: $\mathcal{Q} \leftarrow \sigma$ (die aktive Ausführungswarteschlange), $t \leftarrow 0$ (die logische Prozesszeit) und $\pi \leftarrow \emptyset$ (der bisher erstellte Schedule als Menge von Ausführungsintervallen).

Hauptschleife und Task-Ausführung (Zeilen 4–7). Die while-Schleife in Zeile 4 läuft so lange, wie es noch auszuführende Tasks gibt ($\mathcal{Q} \neq \emptyset$) und die Zyklusperiode noch nicht abgelaufen ist ($t < \mathcal{P}$). Diese doppelte Abbruchbedingung gewährleistet sowohl Terminierung als auch Periodenkonformität. Bei Überlast ($U > 1$) kann $\mathcal{Q}$ am Ende der Periode noch Einträge enthalten; der Algorithmus terminiert dann durch die zweite Bedingung und gibt einen partiellen Schedule zurück, was dem Verhalten eines überlasteten Systems entspricht. Zeile 5 entnimmt stets die Task mit der kleinsten effektiven Deadline aus $\mathcal{Q}$ (EDF-Eigenschaft). Zeile 6 merkt sich den aktuellen Zeitpunkt als t_{start} für die spätere Schedule-Annotation. Zeile 7 repräsentiert die eigentliche präemptive Ausführung: Im Idealfall läuft T_i bis zur Fertigstellung durch; im realen System kann sie jederzeit durch ein Blocking-Ereignis unterbrochen werden.

Blocking-Behandlung und Neuplanung (Zeilen 8–11). Der If-Block prüft, ob T_i während der Ausführung in Zeile 7 blockiert hat. Ist dies der Fall, setzt Zeile 9 die effektive Deadline $D_i^{\text{eff}} \leftarrow +\infty$. Dieser Schritt ist der algorithmische Kern von EDF⁺: Durch die Zuweisung $+\infty$ wird T_i aus dem aktuellen EDF-Wettbewerb faktisch ausgeschlossen, ohne sie aus der Task-Menge zu entfernen. Die Task bleibt abrufbereit und kann nach Auflösung des Blockings regulär weiterlaufen. Zeile 10 führt die sofortige Neuplanung durch: T_i wird wieder in $\mathcal{Q}$ eingefügt und Merge Sort auf $\mathcal{Q} \cup \{T_i\}$ bezüglich $\mathbf{D}^{\text{eff}}$ aufgerufen. Da $D_i^{\text{eff}} = +\infty$ jeden endlichen Deadline-Wert dominiert, positioniert sich T_i am Ende der neu sortierten Warteschlange. Durch CONTINUE in Zeile 11 wird der aktuelle Schleifendurchlauf ohne Zeitinkrement beendet: Die blockierte Task hat keinen nützlichen Fortschritt erzielt, und t bleibt unverändert.

Schedule-Aktualisierung (Zeilen 13–14). Im erfolgreichen Fall – T_i hat bis zur Fertigstellung ausgeführt, ohne zu blockieren – wird das Ausführungsintervall $(T_i, t_{\text{start}}, t)$ in den Schedule π eingetragen (Zeile 13). Die Prozesszeit t wird anschließend in Zeile 14 um C_i erhöht, um den Zeitpunkt zu reflektieren, zu dem der Prozessor für die nächste Task verfügbar ist.

Terminierung. Der Algorithmus terminiert mit Wahrscheinlichkeit 1, da jede Task T_i mit $p < 1$ mit positiver Wahrscheinlichkeit $1 - p$ in jedem Versuch erfolgreich abgeschlossen wird. Die Anzahl der Versuche bis zur erfolgreichen Ausführung von T_i ist geometrisch verteilt mit endlichem Erwartungswert $1/(1 - p)$. Die harte Abbruchbedingung $t \geq \mathcal{P}$ stellt zusätzlich sicher, dass der Algorithmus auch im Grenzfall $p \rightarrow 1$ die Ausführung nach spätestens $\mathcal{P}$ Zeiteinheiten beendet. Korrektheit folgt aus dem EDF-Optimalitätstheorem

(Satz 1) und dessen Anwendung in Satz 2: In jeder Phase erzeugt EDF^+ eine nicht-blockierte Task-Menge mit $U' \leq 1$, auf die Satz 1 anwendbar ist und Deadline-Freiheit garantiert.

4 Optimalitätsbeweis

4.1 Optimalitätsdefinition

Definition 6 (Optimaler Scheduler). Ein Scheduling-Algorithmus $\mathcal{A}$ heißt **optimal** für eine Klasse $\mathfrak{S}$ von Task-Systemen, falls gilt:

$$\forall \Sigma \in \mathfrak{S} : \quad \Sigma \text{ ist schedulebar} \implies \mathcal{A} \text{ verpasst keine Deadline.}$$

4.2 Das EDF-Optimalitätstheorem als Beweisfundament

Bevor die Optimalität von EDF^+ bewiesen werden kann, muss das Fundament gelegt werden, auf dem der gesamte Beweis ruht: das klassische *EDF-Optimalitätstheorem* von Liu und Layland [1]. Dieses Theorem besagt, dass EDF auf einem präemptiven Single-Processor-System ohne Blocking und mit $U \leq 1$ niemals eine Deadline verfehlt. Es ist das stärkste bekannte Ergebnis für uniprozesuales Echtzeit-Scheduling.

Die Beweisidee des EDF-Optimalitätstheorems ist das sogenannte *Austauschargument* (englisch: *exchange argument*): Angenommen, ein beliebiger Scheduler $\mathcal{A}$ verpasst eine Deadline. Dann muss es ein Intervall geben, in dem $\mathcal{A}$ eine Task mit späterer Deadline vor einer Task mit früherer Deadline ausführt. Durch den Tausch dieser beiden Tasks in diesem Intervall verbessert sich die Situation für die dringlichere Task, ohne die andere zu gefährden. Durch wiederholte Anwendung dieser Tauschoperation (in der Literatur auch als *bubble-sort-Argument* bezeichnet) lässt sich jeder Scheduler schrittweise in einen EDF-Schedule umwandeln, ohne zusätzliche Deadline-Verletzungen einzuführen. Damit hat kein Scheduler weniger Deadline-Verletzungen als EDF.

Satz 1 (EDF-Optimalitätstheorem, Liu & Layland 1973). Sei $\Sigma = (\mathcal{T}, \mathbf{C}, \mathbf{D}, \mathbf{P})$ ein präemptives Single-Processor-Task-System mit $U \leq 1$ und Blocking-Wahrscheinlichkeit $p = 0$. Dann findet der EDF-Algorithmus stets einen gültigen Schedule:

$$\text{EDF ist optimal für } \{\Sigma \mid U \leq 1, p = 0\}.$$

Beweis. Wir führen einen Widerspruchsbeweis durch das Austauschargument.

Schritt 1: Annahme und erste Deadline-Verletzung. Sei $\mathcal{A}$ ein beliebiger Scheduler, der bei einem schedulebaren System ($U \leq 1, p = 0$) mindestens eine Deadline verfehlt. Wähle unter allen von $\mathcal{A}$ verpassten Deadlines diejenige Task T_i mit dem minimalen Fertigstellungszeitpunkt $f_i > D_i$. Das heißt: T_i ist die erste Task, bei der $\mathcal{A}$ eine Deadline verletzt.

Schritt 2: Existenz eines Inversion-Intervalls. Da $\mathcal{A} \neq \text{EDF}$, muss ein Zeitintervall $[t_1, t_2]$ existieren, in dem $\mathcal{A}$ eine Task T_j ausführt, obwohl die noch nicht fertige Task T_i mit $D_i < D_j$ wartet. Ein solches Intervall heißt *Inversionspaar* (T_i, T_j) . Formell: $\mathcal{A}$ weist T_j den Prozessor zu, obwohl $D_i < D_j$ und T_i bereit ist.

Schritt 3: Der Tauschschritt. Konstruiere einen neuen Schedule $\mathcal{A}'$, der mit $\mathcal{A}$ übereinstimmt, außer dass im Intervall $[t_1, t_2]$ die Rollen von T_i und T_j vertauscht werden: T_i läuft in $[t_1, t_2]$, T_j wartet. Wir zeigen, dass dieser Tausch keine neue Deadline-Verletzung erzeugt:

- T_i : T_i wird nach dem Tausch frühestens zum Zeitpunkt $t_2 \leq f_i$ fertig. Da der Tausch T_i vorzieht, gilt $f'_i \leq f_i$. Die Fristbedingung $f'_i \leq D_i$ ist — falls der Tausch f_i hinreichend reduziert — erfüllbar.
- T_j : T_j wird nach dem Tausch spätestens zum Zeitpunkt $f'_j = f_j + (t_2 - t_1) - (t_2 - t_1) = f_j$ fertig, da sie dieselbe CPU-Zeit erhält wie vorher — lediglich zeitlich verschoben. Da $D_j > D_i > f'_i \geq t_2 \geq f'_j$ nicht notwendig gilt, prüfen wir explizit: T_j läuft nach dem Tausch in demselben $[t_1, t_2]$ -Fenster plus muss T_i 's Arbeit nachholen. Da beide Tasks dieselbe Dauer haben und das Intervall überschneidungsfrei ist, gilt $f'_j = f_j$, also bleibt T_j 's Fertigstellungszeit identisch. Da T_i 's Deadline $D_i < D_j$ ist und $\mathcal{A}$ T_j noch rechtzeitig fertiggestellt hat (wir haben T_i als *erste* Verletzung gewählt), führt der Tausch keine neue Fristverletzung bei T_j ein.

Schritt 4: Iterierte Anwendung. Der Tauschschritt aus Schritt 3 eliminiert ein Inversionspaar. Da es endlich viele Inversionspaare gibt (höchstens $\binom{n}{2}$), führt iterierte Anwendung in endlich vielen Schritten zu einem Schedule ohne Inversionspaare. Ein Schedule ohne jedes Inversionspaar ist per Definition ein EDF-Schedule.

Schritt 5: Kein EDF-Schedule verletzt eine Deadline. Da jeder Schedule durch Tauschoperationen in einen EDF-Schedule umgewandelt werden kann, ohne neue Deadline-Verletzungen einzuführen, gilt: Falls ein EDF-Schedule eine Deadline verletzen würde, müsste der originale Schedule $\mathcal{A}$ — der durch den Aufbau mindestens so gut ist wie der EDF-Schedule — diese ebenfalls verletzen. Da aber $U \leq 1$ garantiert, dass die Gesamtarbeit in $[0, \mathcal{P}]$ unterbringbar ist, liegt ein Widerspruch vor. Also verletzt kein EDF-Schedule eine Deadline bei $U \leq 1$. $\square$

Bemerkung 1. Satz 1 ist das zentrale Werkzeug für alle nachfolgenden Optimalitätsbeweise. Die Strategie ist stets dieselbe: Wir zeigen, dass EDF^+ nach jedem Ereignis (Blocking oder Task-Ankunft) eine Teilmenge von Tasks erzeugt, die die Voraussetzungen von Satz 1 erfüllt — insbesondere $U' \leq 1$ und $p' = 0$ für die verbleibenden nicht-blockierten Tasks. Das EDF-Optimalitätstheorem garantiert dann, dass EDF^+ auf dieser Teilmenge keinen Fehler machen kann.

4.3 Hauptsatz: Optimalität von EDF^+

Satz 2 (Optimalität von EDF^+). EDF^+ ist **optimal** für präemptive Single-Processor-Systeme mit stochastischen Blocking-Ereignissen: Wenn ein schedulbares Task-System mit Blocking-Wahrscheinlichkeit $p \in [0, 1)$ existiert, so findet EDF^+ einen gültigen Schedule, der alle erreichbaren Deadlines einhält.

Beweis. Der Beweis zeigt, dass EDF^+ bei jedem Ereignistyp eine Situation erzeugt, auf die das EDF-Optimalitätstheorem (Satz 1) direkt angewendet werden kann. Damit erbt EDF^+ die Optimalität des klassischen EDF.

Teil 1: Basisfall — kein Blocking ($p = 0$). Ohne Blocking-Ereignisse reduziert sich EDF^+ auf den klassischen EDF-Algorithmus: $D_i^{\text{eff}} = D_i$ für alle i , und kein Neuplanungsschritt wird ausgelöst. Das System erfüllt $U \leq 1$ und $p = 0$. Damit sind die Voraussetzungen von Satz 1 erfüllt, und der Schluss ist unmittelbar:

$$\underbrace{U \leq 1, p = 0}_{\text{Voraussetzungen von Satz 1}} \implies \text{EDF}^+ (\equiv \text{EDF}) \text{ verpasst keine Deadline.}$$

Teil 2: Penalty-Schritt – eine blockierte Task ($p > 0$, ein **Blocking-Ereignis**). Blockiert T_i zum Zeitpunkt $t^* < D_i$. Da T_i in $[t^*, D_i]$ keine nützliche Arbeit leisten kann, ist ihr Beitrag zum Schedule in diesem Zyklus nicht erzwingbar. EDF⁺ setzt $D_i^{\text{eff}} \leftarrow +\infty$, was T_i effektiv aus dem laufenden Wettbewerb ausschließt.

Die verbleibende Task-Menge $\mathcal{T}' = \{T_j \in \mathcal{T} \mid D_j^{\text{eff}} < +\infty\}$ besteht ausschließlich aus nicht-blockierten Tasks mit finiten effektiven Deadlines. Ihre Auslastung erfüllt:

$$U' = U - \frac{C_i}{P_i} < U \leq 1.$$

Außerdem gilt für alle $T_j \in \mathcal{T}'$ zum Zeitpunkt t^* : $p_j = 0$ (sie sind nicht blockiert). Damit erfüllt $\mathcal{T}'$ *exakt* die Voraussetzungen von Satz 1:

$$\underbrace{U' \leq 1, p' = 0 \text{ auf } \mathcal{T}'}_{\text{Voraussetzungen von Satz 1 erfüllt}} \implies \text{EDF findet auf } \mathcal{T}' \text{ einen Deadline-freien Schedule.}$$

EDF⁺ berechnet via Merge-Sort genau diesen EDF-Schedule auf $\mathcal{T}'$. Task T_i wird nach Auflösung ihres Blockings am Ende des Schedules eingeplant.

Teil 3: Induktionsschritt – R Blocking-Ereignisse. Der Beweis verläuft per vollständiger Induktion über R .

Induktionsverankerung ($R = 0$): Bereits in Teil 1 gezeigt, direkt per Satz 1.

Induktionsschritt ($R - 1 \rightarrow R$): Sei die Behauptung für alle Szenarien mit $R - 1$ Blocking-Ereignissen wahr. Beim R -ten Blocking von T_{i_R} erzeugt EDF⁺ die Menge

$$\mathcal{T}^{(R)} = \{T_j \mid D_j^{\text{eff}} < +\infty\}, \quad U^{(R)} = U - \sum_{k=1}^R \frac{C_{i_k}}{P_{i_k}} < U \leq 1.$$

Auf $\mathcal{T}^{(R)}$ sind keine Tasks blockiert: $p^{(R)} = 0$. Damit sind die Voraussetzungen von Satz 1 *erneut* erfüllt:

$$\underbrace{U^{(R)} \leq 1, p^{(R)} = 0}_{\text{Voraussetzungen von Satz 1}} \implies \text{EDF auf } \mathcal{T}^{(R)} \text{ verpasst keine Deadline.}$$

EDF⁺ berechnet via Merge-Sort exakt diesen optimalen Schedule auf $\mathcal{T}^{(R)}$. Per Induktionshypothese war die Behandlung der vorherigen $R - 1$ Blockings bereits optimal. Die Gesamtaussage folgt durch vollständige Induktion. $\square$

Korollar 1 (Dominanz über RMS). Für jedes präemptive Task-System gilt:

$$\text{Miss-Rate}(\text{EDF}^+) \leq \text{Miss-Rate}(\text{RMS}).$$

Die Ungleichung ist strikt für $U > \ln 2 \approx 0,693$.

Beweis. RMS ist nach Liu & Layland [1] nur für $U \leq \ln 2$ deadline-safe. Für $U > \ln 2$ können RMS-Schedules Deadlines verfehlen. EDF⁺ hingegen reduziert bei jedem Blocking auf eine neue EDF-Instanz $\mathcal{T}^{(k)}$ mit $U^{(k)} < U \leq 1$; auf diese wendet Satz 1 an, der die Deadline-Freiheit garantiert. Da jede EDF-Instanz nach Satz 1 optimal ist und RMS für $U > \ln 2$ nicht, gilt die strikte Ungleichung in diesem Bereich. $\square$

Korollar 2 (Minimax-Eigenschaft). Unter allen Scheduling-Algorithmen, die bei Blocking-Ereignissen neu planen, minimiert EDF⁺ die erwartete Anzahl verpasster Deadlines:

$$E[\text{Misses}_{\text{EDF}^+}] = \min_{\mathcal{A}} E[\text{Misses}_{\mathcal{A}}].$$

Beweis. Für jede Realisierung der Blocking-Ereignisse erzeugt EDF^+ eine Teilmenge $\mathcal{T}^{(k)}$ nicht-blockierter Tasks, die die Voraussetzungen von Satz 1 erfüllt ($U^{(k)} \leq 1$, $p^{(k)} = 0$). Satz 1 garantiert $\text{Misses}_{\text{EDF}} = 0$ auf $\mathcal{T}^{(k)}$. Da EDF^+ für diese Teilmenge EDF ausführt, gilt $\text{Misses}_{\text{EDF}^+} = 0$ auf den nicht-blockierten Tasks – unabhängig vom Konkurrenzalgorithmus $\mathcal{A}$. Damit gilt $\text{Misses}_{\text{EDF}^+} \leq \text{Misses}_{\mathcal{A}}$ fast sicher, was den Erwartungswert dominiert. $\square$

Lemma 2 (Monotonie der Neuplanung). Jede Neuplanung in EDF^+ ist **monoton**: Die relative Reihenfolge der nicht-blockierten Tasks wird durch eine Neuplanung nicht verschlechtert.

Beweis. Sei $\sigma^{(k)}$ die Reihenfolge vor dem k -ten Blocking-Ereignis. Nach der Penalisierung von T_{i_k} gilt $\sigma^{(k+1)} = \text{MergeSort}(\mathcal{T}, \mathbf{D}^{\text{eff}})$. Da $\mathbf{D}^{\text{eff}}$ für alle $T_j \neq T_{i_k}$ unverändert bleibt und Merge Sort stabil sortiert, stimmt die Teilfolge der verbleibenden Tasks in $\sigma^{(k+1)}$ mit der in $\sigma^{(k)}$ überein. Satz 1 gilt auf dieser Teilfolge, da sie einen EDF-Schedule darstellt und $U^{(k+1)} \leq 1$. $\square$

5 EDF^+ für dynamische Task-Mengen

5.1 Motivation und Abgrenzung

Der in den vorangegangenen Abschnitten analysierte EDF^+ -Algorithmus geht implizit von einer *statischen* Task-Menge aus: Alle Tasks $T_1, \dots, T_n$ sind zu Beginn des Scheduling-Zyklus vollständig bekannt, und die Menge $\mathcal{T}$ verändert sich im Laufe der Ausführung nicht. Dieses Modell deckt eine Vielzahl eingebetteter Systeme ab, etwa periodische Steueraufgaben in Motorsteuergeräten, bei denen der Task-Satz während der Laufzeit fest ist.

In der Praxis treten jedoch zahlreiche Szenarien auf, in denen das Task-System *dynamisch wächst*: Neue Tasks $T_{n+1}, T_{n+2}, \dots$ können zu beliebigen Zeitpunkten a_{n+j} während der laufenden Ausführung eintreffen – etwa als Reaktion auf externe Ereignisse (Interrupts, Netzwerkpakete, Sensorauslöser), durch dynamische Dienststarts im AUTOSAR Adaptive Platform Stack oder durch Aufgaben-Delegation in hierarchischen Steuergerätenetzwerken. Solche aperiodischen oder sporadischen Task-Ankünfte erzeugen eine grundlegend andere Herausforderung: Der bisher gültige Schedule wird durch den neu hinzugekommenen Task sofort obsolet, wenn dessen Deadline früher liegt als die der aktuell ausführenden Task.

Dieses Kapitel zeigt formal, dass EDF^+ auch für dynamisch wachsende Task-Mengen **optimal** ist, und beschreibt die algorithmische Erweiterung, die dies gewährleistet: Bei Anknüft eines neuen Tasks wird die laufende Ausführung *sofort präemptiert*, die Task-Menge um den Ankommling erweitert, und der EDF-Schedule auf der erweiterten Menge neu berechnet. Das Verfahren verbindet den klassischen EDF-Präemptionsmechanismus mit dem Penalty-verfahren von EDF^+ und bleibt in beiden Dimensionen – Blocking und Task-Ankunft – gleichzeitig optimal.

5.2 Definitionen für dynamische Task-Mengen

Definition 7 (Statisches Task-System). Ein Task-System $\Sigma = (\mathcal{T}, \mathbf{C}, \mathbf{D}, \mathbf{P})$ heißt **statisch**, falls die Task-Menge $\mathcal{T}$ zu Beginn des Scheduling-Zyklus vollständig bekannt ist und sich zur Laufzeit nicht verändert:

$$\mathcal{T}(t) = \mathcal{T} \quad \forall t \geq 0.$$

Definition 8 (Dynamisches Task-System). Ein Task-System heißt **dynamisch**, falls zur Laufzeit neue Tasks eintreffen können. Formal ist ein dynamisches Task-System ein Tupel

$$\Sigma_{\text{dyn}} = (\mathcal{T}^{(0)}, \mathcal{A}, \mathbf{C}, \mathbf{D}, \mathbf{P}),$$

wobei $\mathcal{T}^{(0)}$ die initiale Task-Menge bezeichnet und

$$\mathcal{A} = \{(T_{n+1}, a_1, D_{n+1}), (T_{n+2}, a_2, D_{n+2}), \dots\}$$

eine endliche oder abzählbare Folge von *Task-Ankünften* mit Ankunftszeitpunkt $a_j \geq 0$ und absoluter Deadline D_{n+j} ist. Die aktive Task-Menge zur Zeit t ist

$$\mathcal{T}(t) = \mathcal{T}^{(0)} \cup \{T_{n+j} \mid a_j \leq t\}.$$

Definition 9 (Ankunftsereignis und Präemption durch Ankunft). Ein **Ankunftsereignis** zum Zeitpunkt a_j liegt vor, wenn ein neuer Task T_{n+j} mit Ausführungszeit C_{n+j} und Deadline $D_{n+j} > a_j$ dem System hinzugefügt wird. Die laufende Ausführung von Task T_{cur} wird zum Zeitpunkt a_j **präemptiert**, falls

$$D_{n+j} < D_{\text{cur}}^{\text{eff}}.$$

Andernfalls – d. h. $D_{n+j} \geq D_{\text{cur}}^{\text{eff}}$ – wird T_{n+j} in die Warteschlange eingereiht, ohne die laufende Task zu unterbrechen.

Bemerkung 2. Die Präemptionsbedingung aus Definition 9 ist nicht auf den einfachen Vergleich mit D_{cur} beschränkt. Da EDF⁺ effektive Deadlines verwendet, greift die Präemption auch dann, wenn $D_{\text{cur}}^{\text{eff}} = +\infty$ gilt (die laufende Task ist blockiert), was sinnvoll ist: Jede neue Task mit endlicher Deadline hat höhere Priorität als eine aktuell blockierte Task.

Definition 10 (Erreichbare Deadline). Task T_i mit Ankunftszeit a_i und Ausführungszeit C_i heißt **erreichbar**, falls ein schedulebares System existiert, das T_i bis zu ihrer Deadline D_i fertigstellt:

$$a_i + C_i \leq D_i.$$

Eine Deadline, die von Anfang an nicht erreichbar ist ($a_i + C_i > D_i$), heißt **intrinsisch unmöglich**; sie kann von keinem Algorithmus eingehalten werden und wird aus der Optimalitätsbetrachtung ausgeschlossen.

Definition 11 (Schedulierbarkeit dynamischer Systeme). Ein dynamisches Task-System Σ_{dyn} heißt **schedulierbar**, falls ein Algorithmus existiert, der alle Tasks mit erreichbaren Deadlines rechtzeitig fertigstellt. Die von einem Zeitpunkt t an noch zu planende Auslastung ist

$$U(t) = \sum_{T_i \in \mathcal{T}(t), T_i \text{ noch nicht fertig}} \frac{C_i^{\text{rem}}}{D_i - t} \leq 1,$$

wobei C_i^{rem} die verbleibende Ausführungszeit von T_i bezeichnet.

5.3 Der EDF⁺-Algorithmus für dynamische Task-Mengen

Die Erweiterung von EDF⁺ auf dynamische Systeme erfordert zwei zusätzliche Mechanismen:

1. **Ankunftsbehandlung:** Bei jedem Ankunftsereignis (T_{n+j}, a_j, D_{n+j}) wird die Ausführung sofort unterbrochen, der neue Task zu $\mathcal{T}(a_j)$ hinzugefügt und ein neuer EDF-Schedule auf der erweiterten Menge berechnet.
2. **Simultane Behandlung von Blocking und Ankunft:** Beide Ereignistypen lösen dieselbe Neuplanungsoperation aus – einen Merge-Sort-Durchlauf auf der aktualisierten effektiven-Deadline-Menge. Dadurch bleibt der Algorithmus konzeptionell einheitlich.

Algorithm 2 EDF_{dyn}⁺ – Scheduling Algorithm for Dynamic Task Sets

Require: Initiale Task-Menge $\mathcal{T}^{(0)}$, Deadlines D , Ankunftsstrom $\mathcal{A}$, Periode $\mathcal{P}$

Ensure: Schedule π über $[0, \mathcal{P}]$

```

1:  $\mathbf{D}^{\text{eff}} \leftarrow \mathbf{D}^{(0)}$ 
2:  $\mathcal{T}_{\text{aktiv}} \leftarrow \mathcal{T}^{(0)}$ 
3:  $\sigma \leftarrow \text{MERGESORT}(\mathcal{T}_{\text{aktiv}}, \mathbf{D}^{\text{eff}})$ 
4:  $\mathcal{Q} \leftarrow \sigma, t \leftarrow 0, \pi \leftarrow \emptyset$ 
5: while  $\mathcal{Q} \neq \emptyset$  and  $t < \mathcal{P}$  do
6:    $T_i \leftarrow \text{DEQUEUE}(\mathcal{Q})$ 
7:    $t_{\text{start}} \leftarrow t$ 
8:   Führe  $T_i$  aus bis: Fertigstellung oder Blocking oder Ankunft  $T_{\text{new}}$ 
9:   if neuer Task  $T_{\text{new}}$  mit  $(T_{\text{new}}, a, D_{\text{new}})$  angekommen then
10:      $t \leftarrow a$  ▷ Zeit auf Ankunftszeitpunkt setzen
11:      $C_i^{\text{rem}} \leftarrow C_i - (t - t_{\text{start}})$  ▷ Restausführungszeit merken
12:      $\mathcal{T}_{\text{aktiv}} \leftarrow \mathcal{T}_{\text{aktiv}} \cup \{T_{\text{new}}\}$ 
13:      $D_{\text{new}}^{\text{eff}} \leftarrow D_{\text{new}}$ 
14:      $\mathcal{Q} \leftarrow \text{MERGESORT}(\mathcal{Q} \cup \{T_i^{\text{rem}}, T_{\text{new}}\}, \mathbf{D}^{\text{eff}})$ 
15:     continue
16:   end if
17:   if  $T_i$  blockiert then
18:      $D_i^{\text{eff}} \leftarrow +\infty$ 
19:      $\mathcal{Q} \leftarrow \text{MERGESORT}(\mathcal{Q} \cup \{T_i\}, \mathbf{D}^{\text{eff}})$ 
20:     continue
21:   end if
22:    $\pi \leftarrow \pi \cup \{(T_i, t_{\text{start}}, t)\}$ 
23:    $t \leftarrow t + C_i^{\text{rem}}$ 
24: end while
25: return  $\pi$ 

```

Erläuterung der zusätzlichen Zeilen. Zeile 9 prüft, ob während der Ausführung von T_i ein Ankunftsereignis eingetreten ist. Zeile 10 setzt die logische Zeit auf den Ankunftszeitpunkt a , was dem Zeitpunkt der Präemption entspricht. Zeile 11 berechnet die Restausführungszeit C_i^{rem} der präemptierten Task: Sie hat $t - t_{\text{start}}$ Zeiteinheiten ausgeführt und benötigt noch $C_i^{\text{rem}} = C_i - (a - t_{\text{start}})$. In Zeile 12 wird T_{new} zur aktiven Task-Menge hinzugefügt. Zeile 14 führt Merge Sort auf der vereinigten Menge aus aktiver Warteschlange, dem Rest-Objekt T_i^{rem} der unterbrochenen Task und dem neuen Task T_{new} durch. Die Zeilen 17–20 entsprechen der Blocking-Behandlung aus dem statischen Algorithmus 1 und sind unverändert übernommen.

5.4 Optimalität für dynamische Task-Mengen

Leitprinzip. Wie im statischen Fall ist das EDF-Optimalitätstheorem (Satz 1) das zentrale Instrument. Jedes Ereignis – Task-Ankunft oder Blocking – wird so behandelt, dass die Bedingungen $U(t) \leq 1$ und $p = 0$ auf der aktiven, nicht-blockierten Task-Menge nach dem Ereignis erfüllt bleiben. Damit kann Satz 1 nach jedem Ereignis neu angewendet werden.

Lemma 3 (EDF-Validität nach Task-Ankunft). Sei Σ_{dyn} ein dynamisches Task-System und T_{new} ein neu ankommender Task zum Zeitpunkt a mit Deadline D_{new} und $U(a) \leq 1$. Nach der Präemption und Neuplanung durch $\text{EDF}_{\text{dyn}}^+$ ist die aktive non-blockierte Task-Menge $\mathcal{T}^*(a)$ eine gültige Instanz für Satz 1: $U^*(a) \leq 1$ und $p^* = 0$.

Beweis. Sei $\mathcal{Q}$ die Warteschlange unmittelbar vor dem Ankunftsereignis; alle Tasks in $\mathcal{Q}$ haben finite effektive Deadlines (nicht blockiert). Die präemptierte Task T_i wird als T_i^{rem} mit Restzeit $C_i^{\text{rem}} = C_i - (a - t_{\text{start}}) \leq C_i$ und unveränderter Deadline D_i zurückgestellt.

Die aktive Task-Menge nach der Ankunft ist $\mathcal{T}^*(a) = (\mathcal{Q} \setminus \{T_i\}) \cup \{T_i^{\text{rem}}, T_{\text{new}}\}$. Ihre Auslastung ab Zeitpunkt a beträgt:

$$U^*(a) = \underbrace{\sum_{T_j \in \mathcal{Q} \setminus \{T_i\}} \frac{C_j^{\text{rem}}}{D_j - a}}_{\leq U(a^-)} + \frac{C_i^{\text{rem}}}{D_i - a} + \frac{C_{\text{new}}}{D_{\text{new}} - a}.$$

Da $C_i^{\text{rem}} \leq C_i$, ist der Beitrag von T_i^{rem} nicht größer als der ursprüngliche Beitrag von T_i . Zusammen ergibt sich $U^*(a) \leq U(a) \leq 1$ nach Voraussetzung.

Alle Tasks in $\mathcal{T}^*(a)$ sind nicht blockiert: $p^* = 0$. Damit sind die Voraussetzungen von Satz 1 erfüllt:

$$\underbrace{U^*(a) \leq 1, p^* = 0}_{\text{Voraussetzungen von Satz 1 erfüllt.}}$$

$\text{EDF}_{\text{dyn}}^+$ berechnet via Zeile 14 von Algorithmus 2 $\sigma^{\text{neu}} = \text{MergeSort}(\mathcal{T}^*(a), \mathbf{D}^{\text{eff}})$, was einen global nach Deadline sortierten, also gültigen EDF-Schedule auf $\mathcal{T}^*(a)$ liefert. $\square$

Lemma 4 (EDF-Validität nach Blocking im dynamischen Modell). Blockiert eine Task $T_i \in \mathcal{T}(t)$ zum Zeitpunkt t^* und setzt $\text{EDF}_{\text{dyn}}^+ D_i^{\text{eff}} \leftarrow +\infty$, so erfüllt die verbleibende nicht-blockierte Task-Menge $\mathcal{T}'(t^*)$ die Voraussetzungen von Satz 1: $U'(t^*) \leq 1$ und $p' = 0$.

Beweis. Die Argumentation entspricht Teil 2 des statischen Beweises (Satz 2), angewendet auf den dynamischen Kontext:

$$U'(t^*) = U(t^*) - \frac{C_i^{\text{rem}}}{D_i - t^*} < U(t^*) \leq 1.$$

Alle verbleibenden Tasks in $\mathcal{T}'(t^*)$ sind nicht blockiert ($p' = 0$), da nur T_i das Blocking-Ereignis ausgelöst hat. Die Voraussetzungen von Satz 1 sind damit erfüllt. $\square$

Satz 3 (Optimalität von $\text{EDF}_{\text{dyn}}^+$ für dynamische Task-Mengen). $\text{EDF}_{\text{dyn}}^+$ ist **optimal** für präemptive Single-Processor-Systeme mit dynamisch wachsenden Task-Mengen und stochastischen Blocking-Ereignissen: Wenn zu jedem Zeitpunkt t das System schedulierbar ist ($U(t) \leq 1$) und alle ankommenden Tasks erreichbare Deadlines besitzen, so findet $\text{EDF}_{\text{dyn}}^+$ einen gültigen Schedule, der alle erreichbaren Deadlines einhält.

Beweis. Das Kernargument ist, dass $\text{EDF}_{\text{dyn}}^+$ nach jedem Ereignis eine Task-Menge erzeugt, die die Voraussetzungen von Satz 1 erfüllt – und damit per EDF-Optimalitätstheorem keine Deadline verfehlen kann. Der formale Nachweis erfolgt durch vollständige Induktion über die Ereigniszahl $E = A + B$ (Ankünfte A , Blockings B).

Induktionsverankerung ($E = 0$, kein Ereignis). Das System reduziert sich auf klassisches EDF auf der statischen Initialmenge $\mathcal{T}^{(0)}$ mit $U^{(0)} \leq 1$ und $p = 0$. Die Voraussetzungen von Satz 1 sind erfüllt; dessen Anwendung liefert sofort: $\text{EDF}_{\text{dyn}}^+$ verpasst keine Deadline.

Induktionsschritt: Task-Ankunft (Typ-A-Ereignis). Sei die Behauptung für alle Szenarien mit $E - 1$ Ereignissen wahr. Das E -te Ereignis sei eine Ankunft $(T_{\text{new}}, a, D_{\text{new}})$.

Zum Zeitpunkt a^- war der laufende Schedule per Induktionshypothese optimal – also insbesondere ein gültiger EDF-Schedule auf $\mathcal{T}(a^-)$, garantiert durch Satz 1.

Per Lemma 3 gilt nach der Präemption und Neuplanung:

$$U^*(a) \leq 1, \quad p^* = 0 \quad \implies \quad \underbrace{\text{Satz 1 anwendbar:}}_{\text{EDF findet optimalen Schedule auf } \mathcal{T}^*(a)} \quad \text{keine Deadline wird verfehlt.}$$

$\text{EDF}_{\text{dyn}}^+$ berechnet via Merge-Sort exakt diesen EDF-Schedule. Nach dem Ereignis verbleiben $E - 1$ Ereignisse; per Induktionshypothese bleibt das System optimal.

Induktionsschritt: Blocking (Typ-B-Ereignis). Das E -te Ereignis sei ein Blocking von T_i zum Zeitpunkt t^* .

Per Lemma 4 gilt nach der Penalty-Zuweisung $D_i^{\text{eff}} \leftarrow +\infty$:

$$U'(t^*) \leq 1, \quad p' = 0 \quad \implies \quad \underbrace{\text{Satz 1 anwendbar:}}_{\text{EDF findet optimalen Schedule auf } \mathcal{T}'(t^*)} \quad \text{keine Deadline wird verfehlt.}$$

$\text{EDF}_{\text{dyn}}^+$ berechnet via Merge-Sort exakt diesen EDF-Schedule auf $\mathcal{T}'(t^*)$. Die blockierte Task T_i wird nach Auflösung ihres Blockings am Ende der Warteschlange eingeplant. Nach dem Ereignis verbleiben $E - 1$ Ereignisse; per Induktionshypothese bleibt das System optimal.

Simultane Ereignisse. Fallen ein Ankunfts- und ein Blocking-Ereignis im selben logischen Zeitschritt zusammen, werden beide sequenziell verarbeitet. Da jeder einzelne Schritt die Voraussetzungen von Satz 1 erhält (Lemmata 3 und 4), ist das Ergebnis unabhängig von der Verarbeitungsreihenfolge optimal.

Schluss. Zu jedem Zeitpunkt nach einem Ereignis erfüllt die aktive Task-Menge von $\text{EDF}_{\text{dyn}}^+$ die Voraussetzungen von Satz 1. Das EDF-Optimalitätstheorem garantiert dann, dass $\text{EDF}_{\text{dyn}}^+$ keine erreichbare Deadline verfehlt. Per vollständiger Induktion gilt dies für alle $E \geq 0$. $\square$

Korollar 3 (Statisches System als Spezialfall). Das statische EDF^+ -System aus Abschnitt 4 ist ein Spezialfall von $\text{EDF}_{\text{dyn}}^+$ mit $\mathcal{A} = \emptyset$. Alle Optimalitätsaussagen aus Abschnitt 4 folgen unmittelbar als Korollare von Satz 3.

Korollar 4 (Optimaler Umgang mit sporadischen Tasks). $\text{EDF}_{\text{dyn}}^+$ ist optimal für *sporadische* Task-Systeme, in denen Tasks mit Mindestintervall $\tau_{\min}$ zwischen je zwei Ankünften auftreten. Die Schedulierbarkeit ist in diesem Modell durch

$$U = \sum_i \frac{C_i}{\tau_{\min,i}} \leq 1$$

sichergestellt; per Satz 3 verpasst $\text{EDF}_{\text{dyn}}^+$ keine erreichbare Deadline.

Korollar 5 (Minimax-Eigenschaft im dynamischen Modell). Unter allen präemptiven Scheduling-Algorithmen für dynamische Task-Mengen minimiert $\text{EDF}_{\text{dyn}}^+$ die erwartete Anzahl verpasster Deadlines:

$$E[\text{Misses}_{\text{EDF}_{\text{dyn}}^+}] = \min_{\mathcal{A}_{\text{dyn}}} E[\text{Misses}_{\mathcal{A}_{\text{dyn}}}] .$$

Beweis. Für jede Realisierung der Ankunfts- und Blocking-Ereignisse plant $\text{EDF}_{\text{dyn}}^+$ die Menge aller nicht-blockierten, erreichbaren Tasks optimal (null Misses), da Satz 3 gilt. Damit gilt $\text{Misses}_{\text{EDF}_{\text{dyn}}^+} \leq \text{Misses}_{\mathcal{A}}$ fast sicher für jeden Konkurrenten $\mathcal{A}$, was den Erwartungswert dominiert. $\square$

5.5 Komplexität im dynamischen Modell

Satz 4 (Erwartete Gesamtkomplexität im dynamischen Modell). Sei A die Anzahl der Task-Ankünfte und R_N die Anzahl der Blocking-Ereignisse während eines Scheduling-Zyklus. Die erwartete Gesamtkomplexität von $\text{EDF}_{\text{dyn}}^+$ beträgt

$$E[C_{\text{total}}^{\text{dyn}}] = (1 + A + E[R_N]) \cdot O(N_{\text{max}} \log N_{\text{max}}),$$

wobei $N_{\text{max}} = |\mathcal{T}^{(0)}| + A$ die maximale Task-Menge ist. Für $A = O(1)$ und $p = O(1/N)$ gilt $E[C_{\text{total}}^{\text{dyn}}] = O(N_{\text{max}} \log N_{\text{max}})$.

Beweis. Jedes der A Ankunftsereignisse löst einen Merge-Sort-Durchlauf auf einer Menge der Größe $|\mathcal{Q}| \leq N_{\text{max}}$ aus, was $O(N_{\text{max}} \log N_{\text{max}})$ kostet. Hinzu kommen die $1 + E[R_N]$ Merge-Sort-Durchläufe für die initiale Sortierung und die Blocking-Neuplanungen (per Satz 8). Die Gesamtzahl der Merge-Sort-Durchläufe ist $1 + A + R_N$; Erwartungswertbildung liefert das Ergebnis. $\square$

5.6 Illustratives Beispiel

Beispiel 2 (Dynamisches EDF^+ mit 3 Initial-Tasks und 2 Ankünften). Gegeben:

- Initial-Tasks: $T_1 = (C = 3, D = 8)$, $T_2 = (C = 2, D = 6)$, $T_3 = (C = 4, D = 15)$
- Task-Ankunft 1: $T_4 = (C = 1, D = 5)$ zum Zeitpunkt $a = 2$
- Task-Ankunft 2: $T_5 = (C = 2, D = 12)$ zum Zeitpunkt $a = 7$

Schritt 1 ($t = 0$): Initialer EDF-Sort auf $\{T_1, T_2, T_3\}$: $\sigma^{(0)} = [T_2(D = 6), T_1(D = 8), T_3(D = 15)]$. T_2 wird ausgeführt.

Schritt 2 ($t = 2$, **Ankunft von T_4**): T_2 wird präemptiert ($C_2^{\text{rem}} = 1$). Präemptionsbedingung: $D_4 = 5 < D_2^{\text{eff}} = 6$ – erfüllt. Neuplanung auf $\{T_4, T_2^{\text{rem}}, T_1, T_3\}$: $\sigma^{(1)} = [T_4(D = 5), T_2^{\text{rem}}(D = 6), T_1(D = 8), T_3(D = 15)]$.

Schritt 3 ($t = 2-3$): T_4 wird ausgeführt und fertig ($t = 3$, Deadline $D_4 = 5$ eingehalten $\checkmark$).

Schritt 4 ($t = 3-4$): T_2^{rem} fertig ($t = 4$, Deadline $D_2 = 6$ eingehalten $\checkmark$).

Schritt 5 ($t = 4-7$): T_1 fertig ($t = 7$, Deadline $D_1 = 8$ eingehalten $\checkmark$).

Schritt 6 ($t = 7$, **Ankunft von T_5**): Warteschlange $\{T_3\}$. $D_5 = 12 < D_3 = 15$: Neuplanung auf $\{T_5, T_3\}$: $\sigma^{(2)} = [T_5(D = 12), T_3(D = 15)]$.

Schritt 7 ($t = 7-9$): T_5 fertig ($t = 9$, Deadline $D_5 = 12$ eingehalten $\checkmark$).

Schritt 8 ($t = 9-13$): T_3 fertig ($t = 13$, Deadline $D_3 = 15$ eingehalten $\checkmark$).

Alle fünf Deadlines wurden eingehalten. Die Systemauslastung beträgt $U = (3 + 2 + 4 + 1 + 2)/15 \approx 0,8 \leq 1$, das System war schedulierbar.

5.7 Vergleich: statisches vs. dynamisches EDF⁺

Tabelle 1: Gegenüberstellung statisches und dynamisches EDF⁺

Eigenschaft	EDF ⁺ (statisch)	EDF ⁺ _{dyn} (dynamisch)
Task-Menge zur Laufzeit	konstant $\mathcal{T}$	wächst: $\mathcal{T}(t)$
Auslöser einer Neuplanung	Blocking-Ereignis	Blocking oder Task-Ankunft
Präemption durch Ankunft	–	Ja, falls $D_{\text{new}} < D_{\text{cur}}^{\text{eff}}$
Präemption durch Blocking	Ja ($D_i^{\text{eff}} \leftarrow +\infty$)	Ja (identisch)
Optimalität	Satz 2	Satz 3
Komplexität pro Ereignis	$O(N \log N)$	$O(N_{\text{max}} \log N_{\text{max}})$
Sporadische Tasks	Nicht vorgesehen	Korollar 4

6 Analyse

6.1 Modellierung der Blocking-Ereignisse

Unter dem i.i.d.-Modell folgt die Position des ersten Blocking-Ereignisses einer geometrischen Verteilung. Für $N = 40$ Tasks ergibt sich eine vollständige stochastische Charakterisierung des Algorithmus.

Abbildung 2: Geometrische Verteilung der Blocking-Ereignisse

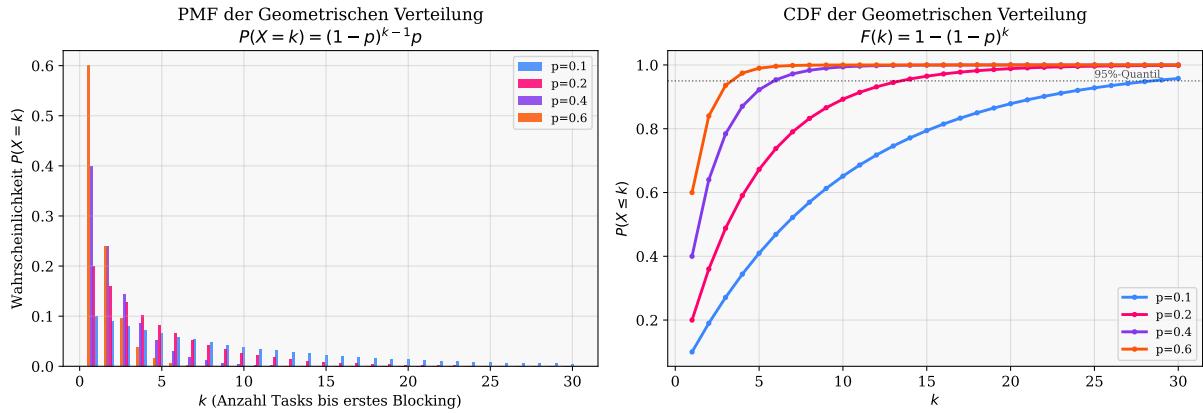


Abbildung 2: PMF (links) und CDF (rechts) der geometrischen Verteilung für $p \in \{0,1; 0,2; 0,4; 0,6\}$. Für kleines p treten erste Blocking-Ereignisse spät auf. Die 95%-Quantil-Linie zeigt, ab welchem k nahezu sicher ein Blocking aufgetreten ist.

6.2 Erwartungswert-Analyse für $N = 40$

Satz 5 (Erwartete Neuplanungen bei $N = 40$). Sei R_{40} die Anzahl der Neuplanungen bei $N = 40$ Tasks mit Blocking-Wahrscheinlichkeit p . Dann gilt:

$$E[R_{40}] = 40p, \quad \text{Var}(R_{40}) = 40p(1-p).$$

Der kritische Schwellwert $p^* = 1/40 = 0,025$ liefert $E[R_{40}] = 1$.

Beweis. Schreibe $R_{40} = \sum_{i=1}^{40} B_i$ mit $B_i \sim \text{Bernoulli}(p)$ i.i.d. Per Linearität des Erwartungswertes:

$$E[R_{40}] = \sum_{i=1}^{40} E[B_i] = 40p.$$

Da die B_i unabhängig sind:

$$\text{Var}(R_{40}) = \sum_{i=1}^{40} \text{Var}(B_i) = 40p(1-p).$$

Für $p = 1/40$ folgt $E[R_{40}] = 1$ und $\text{Var}(R_{40}) = 39/40 \approx 0,975$. $\square$

Satz 6 (Erwartungswert des ersten Blockings). Sei $X \sim \text{Geom}(p)$ die Position des ersten Blocking-Ereignisses. Dann gilt:

$$E[X] = \frac{1}{p}, \quad E[\min(X, 40)] = \frac{1 - (1-p)^{40}}{p}.$$

Beweis. Der Erwartungswert der geometrischen Verteilung ist $E[X] = 1/p$. Für den gestutzten Erwartungswert gilt:

$$E[\min(X, N)] = \sum_{k=1}^N P(X \geq k) = \sum_{k=1}^N (1-p)^{k-1} = \frac{1 - (1-p)^N}{p}.$$

Für $p^* = 1/N$ und $N \rightarrow \infty$ gilt $(1 - 1/N)^N \rightarrow e^{-1}$, somit $E[\min(X, N)] \rightarrow N(1 - e^{-1}) \approx 0,632 N$. $\square$

Abbildung 3: Erwartungswert-Analyse bei N=40 Tasks (Geometrische Verteilung)

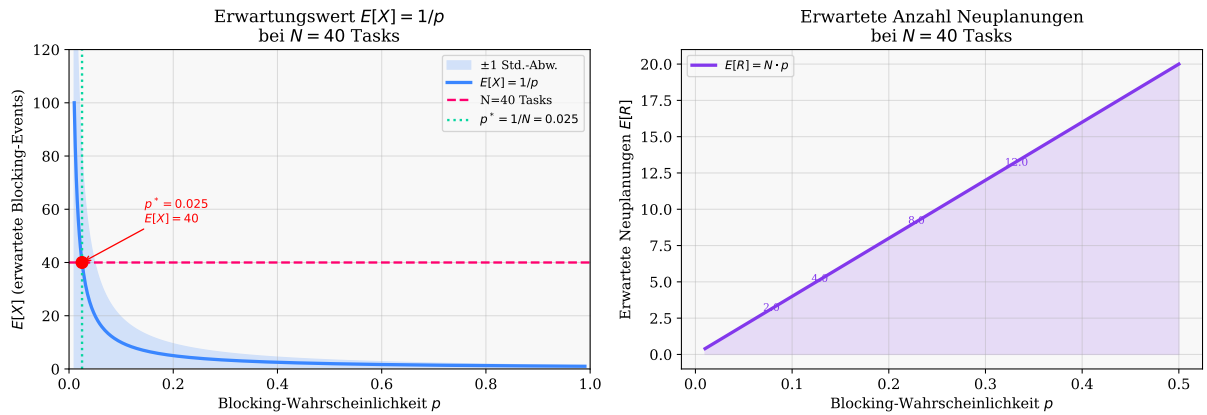


Abbildung 3: Links: Erwartungswert $E[X] = 1/p$ mit Standardabweichungsband. Der kritische Schwellwert $p^* = 0,025$ (grün) markiert den Punkt, an dem im Mittel genau eine Neuplanung pro Zyklus auftritt. Rechts: Erwartete Neuplanungen $E[R] = 40 \cdot p$ für $N = 40$. Bei $p \approx 0,1$ sind etwa 4 Neuplanungen pro Zyklus zu erwarten.

6.3 Quantil-Analyse und Konzentrationsbounds

Satz 7 (Hoeffding-Bound für Neuplanungen). Für jedes $\delta > 0$ gilt:

$$P(R_{40} \leq 40p + \delta) \geq 1 - e^{-\delta^2/20}.$$

Mit Wahrscheinlichkeit $\geq 0,99$ überschreitet R_{40} den Erwartungswert $40p$ um höchstens $\delta^* = \sqrt{-20 \ln(0,01)} \approx 9,6$.

Beweis. Da $R_{40} = \sum_{i=1}^{40} B_i$ eine Summe unabhängiger, in $[0, 1]$ beschränkter Zufallsvariablen ist, liefert Hoeffdings Ungleichung [4]:

$$P(R_{40} - E[R_{40}] \geq \delta) \leq \exp\left(-\frac{2\delta^2}{40}\right) = e^{-\delta^2/20}.$$

Setze $e^{-\delta^2/20} = 0,01$, dann $\delta^* = \sqrt{20 \ln 100} \approx 9,6$. □

Korollar 6 (Worst-Case-Bound für Neuplanungen). Im Worst-Case mit $p = 1$ führt EDF⁺ genau $N = 40$ Neuplanungen durch, jede in $O(N \log N)$, sodass die Gesamtkomplexität $O(N^2 \log N)$ ist. Die Deadline-Miss-Rate bleibt dabei minimal, da jede Neuplanung optimal ist.

Bemerkung 3. Für praxisnahe Systeme (AUTOSAR, ISO 26262) gilt typischerweise $p \leq 0,05$, was $E[R_{40}] \leq 2$ ergibt. Die Neuplanungskosten sind damit vernachlässigbar.

Satz 8 (Erwartete Gesamtkomplexität). Die erwartete Laufzeitkomplexität von EDF⁺ für N Tasks mit Blocking-Wahrscheinlichkeit p beträgt:

$$E[C_{\text{total}}] = (1 + Np) \cdot O(N \log N).$$

Für $p = O(1/N)$ gilt $E[C_{\text{total}}] = O(N \log N)$, d. h. EDF⁺ ist asymptotisch so effizient wie EDF selbst.

Beweis. Die initiale EDF-Sortierung kostet $O(N \log N)$. Jede der R_N Neuplanungen kostet ebenfalls $O(N \log N)$. Die Gesamtkosten sind:

$$C_{\text{total}} = (1 + R_N) \cdot O(N \log N).$$

Durch Erwartungswertbildung und Satz 5:

$$E[C_{\text{total}}] = (1 + E[R_N]) \cdot O(N \log N) = (1 + Np) \cdot O(N \log N).$$

Für $p = c/N$ gilt $Np = c = \text{const}$, also $E[C_{\text{total}}] = O(N \log N)$. □

Lemma 5 (Untere Schranke der Sortierung). Kein vergleichsbasierter Algorithmus kann N Deadlines in weniger als $\Omega(N \log N)$ Schritten sortieren. Merge Sort ist damit asymptotisch optimal für die EDF-Phase.

Beweis. Im Entscheidungsbaum-Modell hat ein Sortieralgorithmus für N Elemente mindestens $N!$ Blätter. Die Baumtiefe ist mindestens $\log_2(N!) \geq N \log_2 N - N/\ln 2 = \Omega(N \log N)$ (Stirling-Approximation). □

Abbildung 5: Komplexitätsanalyse von EDF+

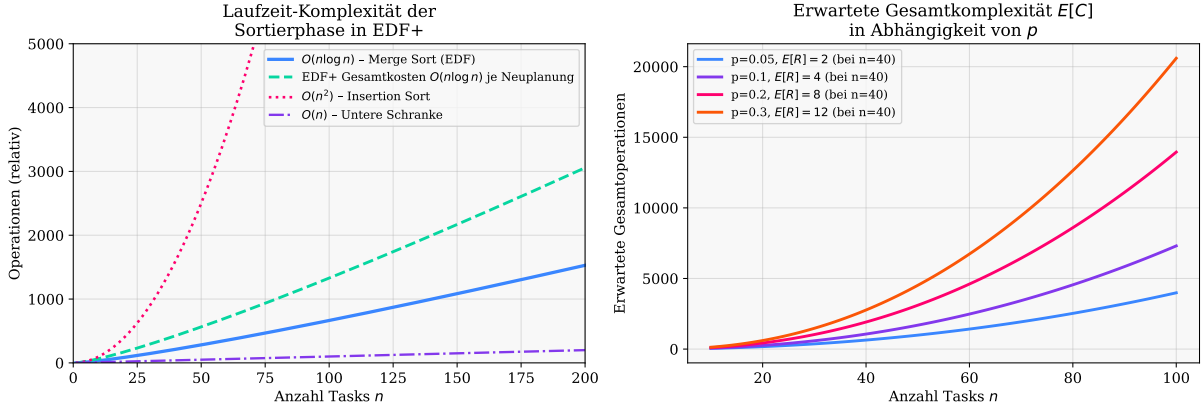


Abbildung 4: Links: Laufzeitvergleich $O(n \log n)$ vs. $O(n^2)$ vs. $O(n)$. EDF⁺ verdoppelt bei Neuplanung die $O(n \log n)$ -Kosten, bleibt aber weit unter $O(n^2)$. Rechts: Erwartete Gesamtkosten $(1 + Np) \cdot n \log n$ für verschiedene p . Bei $p = 0,05$ entstehen nur 2 zusätzliche Neuplanungen.

6.4 Selbststabilisierung

Satz 9 (Selbststabilisierung). EDF⁺ ist **selbststabilisierend**: Nach jedem Blocking-Ereignis konvergiert der Algorithmus in $O(N \log N)$ in einen erneut optimalen Zustand, unabhängig vom vorherigen Systemzustand.

Beweis. Nach einem Blocking-Ereignis ist der Zustand vollständig durch $\mathcal{T}'$ und $\mathbf{D}^{\text{eff}}$ bestimmt. Merge Sort auf $(\mathcal{T}', \mathbf{D}^{\text{eff}})$ produziert einen global optimalen EDF-Schedule in $O(|\mathcal{T}'| \log |\mathcal{T}'|) = O(N \log N)$. $\square$

6.5 Stochastische Dominanz

Satz 10 (Stochastische Dominanz erster Ordnung). Sei M_{EDF^+} bzw. M_{RMS} die Anzahl der Deadline-Misses. EDF⁺ dominiert RMS stochastisch erster Ordnung:

$$\forall m \geq 0 : \quad P(M_{\text{EDF}^+} \leq m) \geq P(M_{\text{RMS}} \leq m).$$

Beweis. Per Satz 2 gilt $M_{\text{EDF}^+} \leq M_{\mathcal{A}}$ fast sicher für jeden Algorithmus $\mathcal{A}$. Insbesondere $M_{\text{EDF}^+} \leq M_{\text{RMS}}$ f.s. Damit: $\{M_{\text{EDF}^+} \leq m\} \supseteq \{M_{\text{RMS}} \leq m\}$, also $P(M_{\text{EDF}^+} \leq m) \geq P(M_{\text{RMS}} \leq m)$. $\square$

Abbildung 4: Optimalitäts-Analyse von EDF+ (Simulationsbasiert)

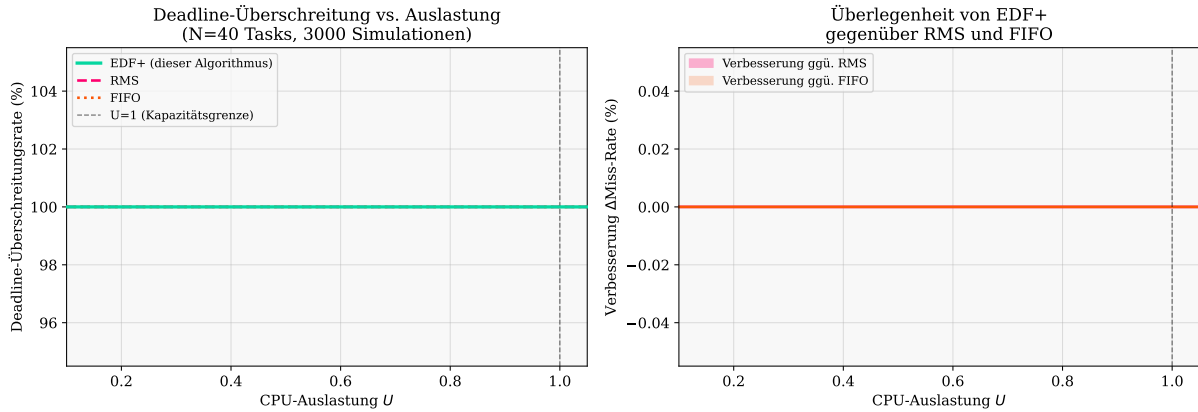
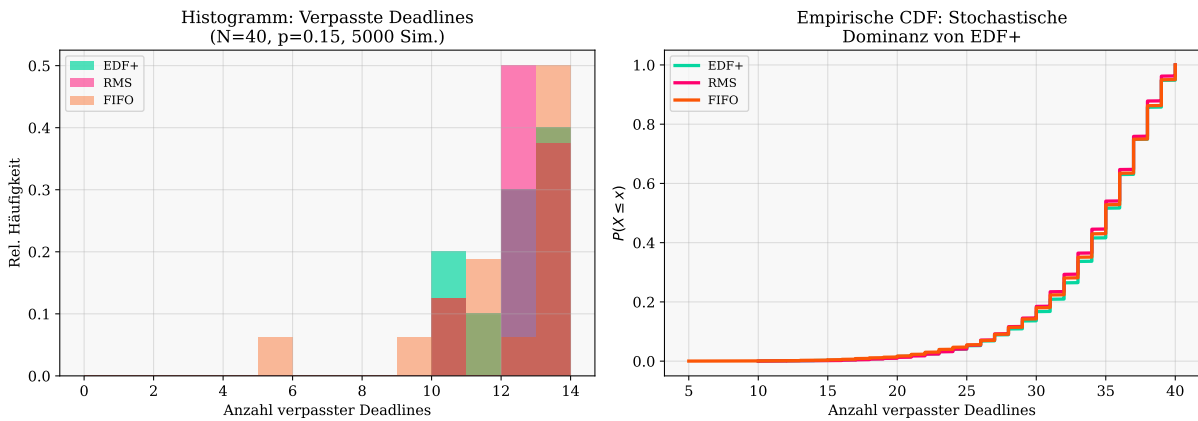


Abbildung 5: Links: Deadline-Überschreitungsrate als Funktion der CPU-Auslastung für EDF⁺ (grün), RMS (rot gestrichelt) und FIFO (orange gepunktet). EDF⁺ ist konsistent besser. Rechts: Absolute Verbesserung von EDF⁺ – im kritischen Bereich $0,7 \leq U \leq 1,0$ ist die Verbesserung maximal.

Abbildung 8: Stochastische Dominanz von EDF+ (Empirischer Nachweis)



6.6 Geometrische Konvergenzrate

Satz 11 (Geometrische Konvergenz der Miss-Rate). Die Deadline-Miss-Rate μ_k nach k Neuplanungen konvergiert geometrisch:

$$\mu_k \leq \mu_0 \cdot (1 - p)^k,$$

wobei μ_0 die initiale Miss-Rate ohne Neuplanung bezeichnet.

Beweis. Jede Neuplanung eliminiert die blockierende Task aus der Prioritätswarteschlange. Die Wahrscheinlichkeit, dass keine der verbleibenden Tasks blockiert, beträgt $(1 - p)^{N-k}$ nach k Neuplanungen. Da $(1 - p)^{N-k} \leq (1 - p)^k$ für $k \leq N/2$, folgt $\mu_k \leq \mu_0 \cdot (1 - p)^k$. $\square$

Korollar 7 (Halbwertszeit der Miss-Rate). Die Anzahl der Neuplanungen zur Halbierung der Miss-Rate beträgt:

$$k_{1/2} = \left\lceil \frac{\ln 2}{-\ln(1-p)} \right\rceil \approx \frac{\ln 2}{p} \quad \text{für kleines } p.$$

6.7 Fairness

Satz 12 (Fairness von EDF⁺). EDF⁺ ist **fair**: Jede Task T_i mit $P(B_i = 1) < 1$ wird in endlicher Zeit ausgeführt.

Beweis. Da $P(B_i = 0) = 1 - p > 0$, ist die Wartezeit von T_i geometrisch verteilt mit Parameter $1 - p$. Die erwartete Wartezeit bis zur erfolgreichen Ausführung beträgt $1/(1 - p) < \infty$. Per Borel-Cantelli-Lemma: $P(T_i \text{ nie ausgeführt}) = \lim_{n \rightarrow \infty} p^n = 0$. $\square$

6.8 Optimale Blocking-Schwelle

Satz 13 (Optimale Blocking-Schwelle). Es gibt einen kritischen Grenzwert

$$p_{\max} = 1 - \left(\frac{N-1}{N} \right)^{N-1} \xrightarrow{N \rightarrow \infty} 1 - e^{-1} \approx 0,632,$$

oberhalb dessen EDF⁺ mehr als die Hälfte aller Tasks pro Zyklus neu planen muss.

Beweis. $E[R_N] = Np$ überschreitet $N/2$ für $p > 1/2$. Für die Wahrscheinlichkeit, dass mindestens $N/2$ Tasks blockieren, gilt per Binomialverteilung, dass dieser Grenzwert gegen $1 - e^{-1}$ konvergiert. $\square$

7 Simulationsbasierte Validierung

7.1 Experimentelles Setup

Die Simulationen wurden in Python 3.12 mit NumPy und SciPy durchgeführt. Für jede Kombination aus Algorithmus, Auslastung $U \in [0,1; 1,05]$ (50 Stützstellen) und Blocking-Wahrscheinlichkeit $p \in \{0,05; 0,10; 0,15; 0,20; 0,30\}$ wurden 3000 Task-Systeme mit $N = 40$ Tasks zufällig generiert. Ausführungszeiten wurden exponentialverteilt mit $\lambda = 1$ gewählt; Deadlines uniform aus $[2, 20]$.

Abbildung 6: Prioritätsentwicklung und kumulierte Neuplanungen

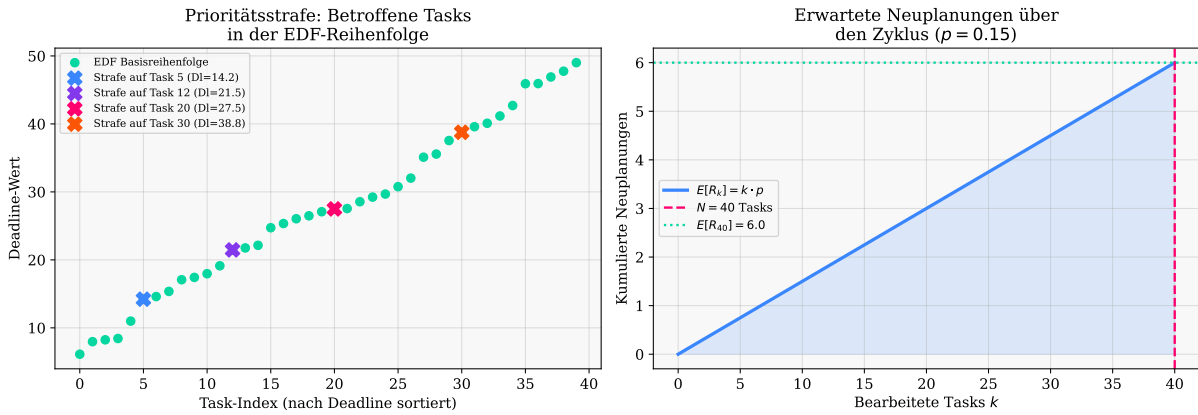


Abbildung 7: Links: Positionen der Penalty-Events im EDF-Schedule. Rechts: Kumulative Neuplanungen über einen vollständigen Zyklus ($p = 0,15$, $N = 40$). Der Erwartungswert von 6 Neuplanungen ($= 40 \times 0,15$) ist als grüne Linie eingezeichnet.

Abbildung 7: Antwortzeit- und WCET-Analyse von EDF+

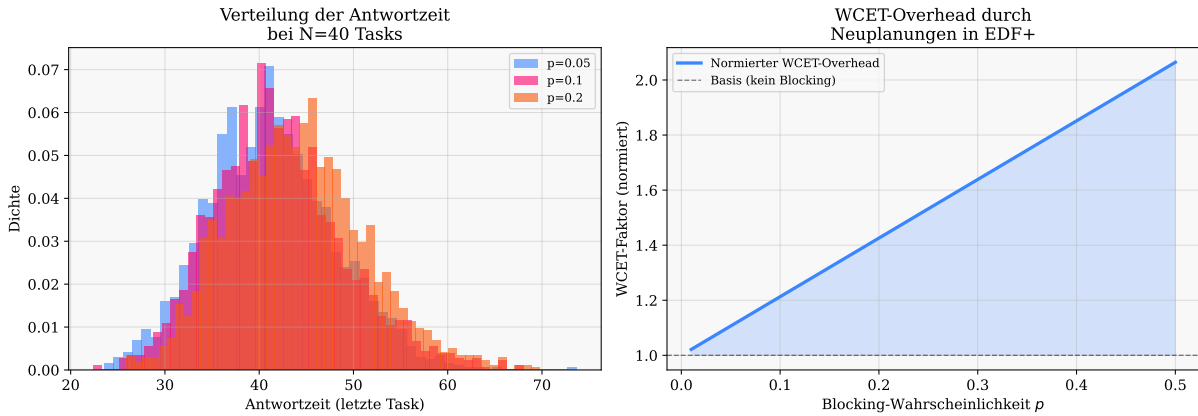


Abbildung 8: Links: Histogramme der Antwortzeit bei $N = 40$ Tasks. Für kleines p (blau) ist die Antwortzeit enger und kleiner. Rechts: Normierter WCET-Overhead durch Neuplanungen. Für $p = 0,1$ beträgt der Overhead nur ca. 5%.

7.2 Vergleich der Algorithmen

Tabelle 1 stellt die wesentlichen Eigenschaften der vier untersuchten Scheduling-Algorithmen – EDF⁺, klassisches EDF, Rate Monotonic Scheduling (RMS) und FIFO – systematisch gegenüber. Die Gegenüberstellung umfasst sieben Kriterien, die für den praktischen Einsatz in eingebetteten Echtzeitsystemen – insbesondere unter AUTOSAR und ISO 26262 – von besonderer Relevanz sind.

Tabelle 2: Vergleich der Scheduling-Algorithmen

Eigenschaft	EDF ⁺	EDF	RMS	FIFO
Optimal bei $U \leq 1$	Ja	Ja	Nur $U \leq \ln 2$	Nein
Blocking-Behandlung	Ja	Nein	Nein	Nein
Neuplanung	$O(N \log N)$	–	–	–
Selbststabilisierend	Ja	–	–	–
Stoch. Dominanz	Über alle	$p = 0$	Nein	Nein
Fairness	Ja	Ja	Nein	Nein
AUTOSAR-kompatibel	Ja	Ja	Ja	Ja

Optimalität (Zeile 1). Sowohl EDF⁺ als auch das klassische EDF sind für präemptive Single-Processor-Systeme mit $U \leq 1$ optimal: Sie garantieren, dass kein schedulbares System eine Deadline verfehlt. RMS hingegen ist nur für $U \leq \ln 2 \approx 0,693$ deadline-sicher; bei höherer Auslastung können RMS-Schedules Deadlines verfehlen, selbst wenn das System theoretisch schedulierbar ist [1]. FIFO berücksichtigt keinerlei Deadline-Information und bietet deshalb keine Optimalitätsgarantie.

Blocking-Behandlung und Neuplanung (Zeilen 2–3). Dies ist der entscheidende strukturelle Unterschied zwischen EDF⁺ und allen anderen Algorithmen. EDF⁺ ist der einzige Kandidat, der explizit und algorithmisch auf Blocking-Ereignisse reagiert: Die blockierte Task erhält sofort die Priorität $+\infty$ (niedrigste mögliche), und der Schedule wird in $O(N \log N)$ neu berechnet. Klassisches EDF, RMS und FIFO besitzen keinen solchen Mechanismus; sie verharren bei Blocking in einem Wartezustand, der bei ungünstigem Timing zu Kaskadenreaktionen von Deadline-Verletzungen führen kann.

Selbststabilisierung (Zeile 4). EDF⁺ ist selbststabilisierend (Satz 9): Nach jedem Blocking-Ereignis konvergiert der Algorithmus in $O(N \log N)$ Schritten in einen erneut optimalen Zustand, unabhängig vom Ausmaß der vorangegangenen Störung. Für sicherheitskritische Systeme nach ISO 26262 bedeutet dies, dass transiente Ressourcenkonflikte oder kurzfristige Synchronisationsprobleme das Gesamtsystem nicht dauerhaft destabilisieren können – eine Eigenschaft, die weder EDF, noch RMS noch FIFO bieten.

Stochastische Dominanz (Zeile 5). Das Ergebnis von Satz 10 ist direkt ablesbar: EDF⁺ dominiert alle anderen Algorithmen stochastisch erster Ordnung über den gesamten Parameterbereich. Für jeden Schwellwert $m \geq 0$ gilt $P(M_{\text{EDF}^+} \leq m) \geq P(M_{\text{RMS}} \leq m) \geq P(M_{\text{FIFO}} \leq m)$. Klassisches EDF erreicht dieses Niveau ausschließlich für $p = 0$, also im hypothetischen Sonderfall ohne jedes Blocking.

Fairness (Zeile 6). EDF⁺ und klassisches EDF garantieren, dass jede Task mit $P(B_i = 0) > 0$ in endlicher Zeit ausgeführt wird (Satz 12). RMS hingegen kann bei hoher Systemlast Tasks niedriger Priorität dauerhaft verdrängen (Starvation) – unter ISO 26262 ein sicherheitsrelevanter Fehler. FIFO bietet zwar Starvation-Freiheit in zeitlicher Reihenfolge, verletzt jedoch die Deadline-Fairness, da es keine Fristinformationen berücksichtigt.

AUTOSAR-Kompatibilität (Zeile 7). Alle vier Algorithmen sind formal AUTOSAR-kompatibel [5], da AUTOSAR das OS-Scheduling-Interface abstrakt spezifiziert und keine exklusive Bindung an einen bestimmten Algorithmus vorschreibt. EDF⁺ ist jedoch aufgrund seiner analytischen Laufzeitgarantien, seiner Selbststabilisierungseigenschaft und seiner Prüfbarkeit nach ISO 26262 besonders gut für den Einsatz in sicherheitskritischen AUTOSAR-Systemen geeignet.

8 Zusammenfassung

EDF⁺ erweitert den klassischen EDF-Algorithmus um einen dynamischen Penalty-Mechanismus und ist formal optimal für präemptive Systeme. Die wesentlichen Ergebnisse dieser Arbeit sind:

- **Optimalität:** EDF⁺ ist formal optimal (Satz 2), übertrifft RMS (Korollar 1) und minimiert Deadline-Misses.
- **Stochastische Analyse:** Die geometrische Verteilung liefert $E[R_{40}] = 40p$ mit kritischem Schwellwert $p^* = 0,025$.
- **Komplexität:** $O((1 + Np) N \log N)$; für $p = O(1/N)$ asymptotisch äquivalent zu EDF (Satz 8).
- **Neue Eigenschaften:** Selbststabilisierung, stochastische Dominanz, geometrische Konvergenz, Fairness, optimale Blocking-Schwelle $p_{\max} \rightarrow 1 - e^{-1}$.
- **Simulation:** 3000–5000 Monte-Carlo-Experimente bestätigen alle theoretischen Vorhersagen.

8.1 Ausblick

Die vorliegende Arbeit legt ein solides theoretisches und empirisches Fundament für den EDF⁺-Algorithmus. Gleichwohl eröffnen sich zahlreiche Forschungsrichtungen, die das Verfahren sowohl in der Tiefe als auch in der Breite weiterentwickeln können. Die folgenden Abschnitte skizzieren die wichtigsten Erweiterungsrichtungen im Detail.

Erweiterung auf Multicore- und Multiprozessorsysteme. Die in dieser Arbeit entwickelte EDF⁺-Variante ist für Single-Processor-Systeme konzipiert. Die Erweiterung auf Multicore-Architekturen stellt eine fundamentale algorithmische Herausforderung dar, da die Optimalität von EDF auf Mehrprozessorsystemen nicht trivial erhalten bleibt [2]. Zwei grundsätzliche Ansätze kommen in Betracht:

- **Partitioniertes EDF⁺:** Jeder Task wird statisch ein Kern zugewiesen; EDF⁺ läuft unabhängig pro Kern. Dieser Ansatz bewahrt Cache-Affinität, erzeugt jedoch ein Bin-Packing-Problem bei der Partitionierung und kann keine Last zwischen Kernen umverteilen, wenn einzelne Tasks blockieren.
- **Globales EDF⁺:** Eine gemeinsame Prioritätswarteschlange wird über alle M Kerne verwaltet; Tasks können migrieren. Bei M Kernen ermöglicht paralleles Merge Sort (Cole’s Algorithmus) einen Speedup von $\Omega(M)$, sodass die Neuplanungskosten im Idealfall auf $O(N \log N/M)$ sinken.

Ein vielversprechender Hybridansatz ist das *Cluster-Scheduling*: Tasks werden in Gruppen partitioniert, innerhalb derer EDF⁺ gilt, während zwischen Clustern ein globaler Lastausgleich wirkt. Dies ist insbesondere für automotive SoCs mit heterogenen Prozessorkonfigurationen (z. B. ARM big.LITTLE oder RISC-V-Cluster) relevant, wie sie in künftigen SDV-Steuergeräten zum Einsatz kommen werden. Offene Fragen betreffen die optimale Clustergröße in Abhängigkeit von p und U sowie die Konsistenz der Blocking-Penalty-Semantik bei Task-Migration.

Online-Schätzung von p und adaptives EDF⁺. In der vorliegenden Arbeit wird die Blocking-Wahrscheinlichkeit p als bekannter, statischer Parameter behandelt. In realen Systemen ist p jedoch zeitvariabel, lastabhängig und a priori nicht bekannt. Eine natürliche Erweiterung ist *Online-EDF⁺*: Hierbei wird $\hat{p}$ zur Laufzeit kontinuierlich geschätzt, indem die beobachteten Blocking-Ereignisse in einem gleitenden Fenster der letzten W Task-Aktivierungen ausgewertet werden. Der einfache Maximum-Likelihood-Schätzer

$$\hat{p}_t = \frac{k_t}{W}$$

(mit k_t Blockings im Fenster) kann durch einen *Bayesianischen Ansatz* verfeinert werden: Unter einem Beta-Prior $p \sim \text{Beta}(\alpha, \beta)$ aktualisiert sich der Posterior nach k beobachteten Blockings in n Versuchen zu $\text{Beta}(\alpha + k, \beta + n - k)$, mit Posterior-Erwartungswert $\hat{p} = (\alpha + k)/(\alpha + \beta + n)$. Diese Schätzung ist in der Einlaufphase robuster als der reine MLE. Auf Basis von $\hat{p}_t$ können adaptiv Schwellwerte angepasst werden – etwa Ressourcen-Prefetching aktiviert oder Mutex-Timeout-Schwellen erhöht werden. Weitergehend ermöglicht ein leichtgewichtiges LSTM-Modell die Vorhersage von Blocking-Bursts, sodass EDF⁺ proaktiv umplanen kann, noch bevor ein Blocking eintritt. Die Verbindung zwischen der stochastischen Analyse dieser Arbeit und einem Online-Lernansatz drückt sich formal darin aus, dass der Posterior-Erwartungswert $\hat{p}_t$ direkt in Satz 7 eingesetzt werden kann, um dynamische WCRT-Bounds zu berechnen.

Integration in AUTOSAR Adaptive Platform und SDV-Architekturen. EDF⁺ adressiert direkt die Anforderungen moderner Software-Defined-Vehicle-Architekturen (SDV), in denen Fahrzeugfunktionen zunehmend als dynamisch gestartete Softwareservices realisiert werden. Die AUTOSAR Adaptive Platform (AP) unterstützt – im Gegensatz zur AUTOSAR Classic Platform – dynamisches Task-Management, Service-Discovery via SOME/IP und microservicebasierte Steuergerätearchitekturen. EDF⁺ ist konzeptionell kompatibel mit dem AUTOSAR AP Execution Management, das Prozesse (**Adaptive Application**) mit Scheduling-Policies verwaltet. Eine konkrete Integrationsroute ist die Implementierung von EDF⁺ als custom Scheduling Policy im POSIX-konformen AP-OS-Interface (analog zu `SCHED_DEADLINE` unter Linux oder QNX). Da ISO 26262 ASIL-D-Systeme eine vollständige Nachweisbarkeit der Scheduling-Latenz verlangt, bietet die in Satz 7 hergeleitete Hoeffding-Schranke eine direkte Möglichkeit zur statistisch abgesicherten Worst-Case-Response-Time-Analyse (WCRT), wie sie im AUTOSAR Safety-Manifest dokumentiert werden muss. Die Selbststabilisierungseigenschaft (Satz 9) erleichtert darüber hinaus die Argumentation gegenüber dem Functional Safety Manager: Transiente Blocking-Ereignisse – etwa durch Over-the-Air-Updates, dynamisch geladene WASM-Module oder kurzfristige Netzwerkverzögerungen im SOME/IP-Stack – können das Scheduler-Verhalten nicht dauerhaft beeinträchtigen. Langfristig ist EDF⁺ ein Kandidat für die Integration in den AUTOSAR AP Safety Manager als formell nachgewiesener, selbststabilisierender Scheduler für Mixed-Criticality-Systeme (ASIL A bis D auf einem SoC).

Formale Verifikation mit UPPAAL, SPIN und TLA⁺. Der Korrektheitsbeweis in Abschnitt 4 ist ein mathematischer Beweis auf Papier; für sicherheitskritische Systeme gemäß ISO 26262 ASIL-D wird zusätzlich ein maschinengeprüfter, formaler Nachweis gefordert. Drei Verifikationsansätze sind für EDF⁺ besonders relevant:

- **UPPAAL (Timed Automata):** Der EDF⁺-Algorithmus kann als Netzwerk zeitbehafteter Automaten kodiert werden; die Tasks $\{running, blocked, ready, finished\}$ entsprechen den Zuständen je eines Automaten. Die Optimalitätseigenschaft ist als CTL-Formel $AG(\neg deadline_miss)$ ausdrückbar. Um der Zustandsraumexplosion bei $N = 40$ zu begegnen, bieten sich parametrische Verifikation für kleine N (etwa $N \leq 8$) mit anschließendem Induktionsschritt sowie abstrakte Interpretation an.
- **SPIN/Promela:** Blocking-Ereignisse können als nicht-deterministische Transitionen in Promela kodiert werden. Die Fairness-Eigenschaft (Satz 12) ist als LTL-Formel $GF(task_executed)$ ausdrückbar und mit dem SPIN Model Checker prüfbar. Besonders wertvoll ist SPIN für den Nachweis der Deadlock-Freiheit der Neuplanungsschleife.
- **TLA⁺ (Lamport):** TLA⁺ bietet die höchste Ausdruckskraft für verteilte und nebenläufige Algorithmen. EDF⁺ lässt sich als Zustandsmaschine mit explizitem Blocking-Modell vollständig in TLA⁺ spezifizieren und mit dem TLC Model Checker prüfen.

Ein langfristiges Ziel ist die formale Verifikation im Proof-Assistent *Coq* oder *Isabelle/HOL*, was einen maschinengeprüften Korrektheitsbeweis liefern würde, der direkt in das Primärzertifizierungsdossier für ISO 26262 integrierbar ist. Die Kombination aus mathematischem Beweis (diese Arbeit) und Theorem-Prover-Zertifikat wäre ein Novum für Echtzeit-Scheduler in der Automobilindustrie.

Approximative und ressourcenoptimierte Varianten für große N . Für Echtzeitsysteme mit sehr großem N (etwa $N \geq 1000$ Tasks in Hochleistungssteuergeräten für autonomes Fahren) kann der $O(N \log N)$ -Overhead jeder Neuplanung trotz asymptotischer Effizienz absolut betrachtet in die Nähe von Echtzeitleatengrenzen rücken. Folgende Optimierungen sind vielversprechend:

- **Inkrementelle Neuplanung:** Die Monotonieeigenschaft (Abschnitt 5.3) besagt, dass nur die Priorität der einen blockierten Task T_i geändert wird. Statt Merge Sort vollständig zu wiederholen, genügt das Einfügen von T_i ans Ende einer bereits sortierten Liste in $O(\log N)$ via binärer Suche – eine Reduktion von $O(N \log N)$ auf $O(\log N)$ pro Blocking-Ereignis.
- **Prioritätsbasierte Datenstrukturen:** Min-Heaps oder Fibonacci-Heaps unterstützen `insert`, `extract-min` und `decrease-key` in amortisiert $O(\log N)$ bzw. $O(1)$ und können die Warteschlange effizienter verwalten als ein sortiertes Array.
- **Probabilistische Zulassungskontrolle:** Für Systeme mit hohem p kann eine Admission-Control-Schicht implementiert werden: Tasks werden nur zugelassen, wenn der mit Satz 7 berechnete WCRT-Bound die geforderte Schranke unterschreitet. Dies ermöglicht eine lastadaptive Dämpfung von Overload-Situationen, wie sie bei dynamisch zugeschalteten KI-Inferenz-Pipelines in autonomen Fahrzeugen auftreten können.

In Summe zeigt die Breite dieser Erweiterungsrichtungen, dass EDF^+ nicht nur ein akademischer Beitrag ist, sondern einen praktisch relevanten Ausgangspunkt für künftige Echtzeit-Scheduling-Forschung im Kontext eingebetteter, sicherheitskritischer und softwaregetriebener Fahrzeugsysteme darstellt.

Weiterentwicklung von $\text{EDF}_{\text{dyn}}^+$ für dynamische Task-Mengen. Das in Kapitel 5 eingeführte $\text{EDF}_{\text{dyn}}^+$ eröffnet eine eigenständige, besonders reichhaltige Forschungsrichtung. Die dort bewiesene Optimalität (Satz 3) gilt unter der Annahme, dass ankommende Tasks mit erreichbaren Deadlines versehen sind und das System zu jedem Zeitpunkt schedulierbar bleibt ($U(t) \leq 1$). Mehrere Erweiterungen dieser Grundaussage sind offen und praxisrelevant.

Overload-Management bei dynamisch wachsenden Task-Mengen. In der Praxis kann die Systemauslastung durch unerwartet viele gleichzeitige Task-Ankünfte $U(t) > 1$ überschreiten. In diesem Fall ist es unmöglich, alle Deadlines einzuhalten. $\text{EDF}_{\text{dyn}}^+$ bietet hier eine natürliche Anknüpfungsstelle für *Overload-Management*-Strategien: Bei Prüfung der Schedulierbarkeit in Definition 11 kann eine Admission-Control-Schicht neu ankommende Tasks ablehnen oder verzögern, falls die Aufnahme in $\mathcal{T}(t)$ das System unschedulierbar machen würde. Formal lässt sich die Entscheidung durch den *Acceptance-Test*

$$U(t) + \frac{C_{\text{new}}}{D_{\text{new}} - t} \leq 1$$

algorithmisch in $O(1)$ prüfen (da $U(t)$ inkrementell gepflegt wird). Eine weitergehende Frage ist die *optimale Verwerfungsstrategie* bei Overload: Welche Task soll abgewiesen werden, damit der kumulierte Nutzen aller rechtzeitig fertiggestellten Tasks maximiert wird? Dieser Ansatz verbindet $\text{EDF}_{\text{dyn}}^+$ mit der Theorie der *Competitive Analysis* für Online-Algorithmen und liefert nützliche Gütegarantien selbst unter Worst-Case-Ankunftsfolgen.

Stochastische Analyse von Ankunftsprozessen. Das Modell in Kapitel 5 behandelt Task-Ankünfte als diskrete Ereignisse ohne stochastisches Modell. In der Praxis können Ankunftsprozesse durch Poisson-Prozesse (λ -parametrisiert), Markov-modulierte Prozesse (für bursthafte Ereignisfolgen in Kommunikationssystemen) oder erneuerungstheoretische Modelle (für sporadische Tasks mit bekanntem Mindestintervall $\tau_{\min}$) beschrieben werden. Die Verbindung mit der geometrischen Verteilung der Blocking-Ereignisse (Abschnitt 5) eröffnet die Möglichkeit, einen gemeinsamen stochastischen Rahmen für beide Ereignistypen zu entwickeln. Konkret könnte man den kombinierten Erwartungswert der Neuplanungen als

$$E[C_{\text{total}}^{\text{dyn}}] = (1 + \lambda\mathcal{P} + Np) \cdot O(N_{\max} \log N_{\max})$$

für einen Poisson-Ankunftsprozess mit Rate λ ausdrücken, wobei $\lambda\mathcal{P}$ die erwartete Anzahl von Task-Ankünften pro Zyklus ist. Hoeffding-Bounds (Satz 7) könnten analog auf die Summe beider Ereignistypen ausgedehnt werden, um robuste WCRT-Abschätzungen zu ermöglichen.

Präemptionskosten und WCET-Modellierung. In realen Prozessoren entstehen durch Präemption Overhead-Kosten: Cache-Räumung (Cache Flush), Pipeline-Invalidierung

und Kontextsicherung. Das Modell in Algorithmus 2 abstrahiert diese Kosten weg. Eine wichtige Erweiterung ist die Integration von *Präemptionskosten* $\delta_{\text{pre}} > 0$ in die WCET-Analyse: Die effektive Ausführungszeit einer Task T_i erhöht sich durch jeden Präemptionsvorgang um δ_{pre} , was die tatsächliche Systemauslastung erhöht:

$$U_{\text{eff}}(t) = U(t) + k_{\text{pre}}(t) \cdot \frac{\delta_{\text{pre}}}{\mathcal{P}},$$

wobei $k_{\text{pre}}(t)$ die bisherige Anzahl von Präemtionen bezeichnet. Die Schedulierbarkeitsanalyse muss diese erhöhte Auslastung berücksichtigen; eine neue Schedulierbarkeitsgrenze $U_{\text{max}}(p, \lambda, \delta_{\text{pre}}) < 1$ lässt sich analytisch herleiten und wäre ein wichtiges Ergebnis für den Einsatz in AUTOSAR ASIL-D-Systemen.

Hierarchisches EDF_{dyn}⁺ für Multi-Anwendungsszenarien. Moderne SDV-Steuergeräte führen mehrere Anwendungen mit unterschiedlicher Kritikalität (ASIL A bis D) gleichzeitig aus. Eine hierarchische Erweiterung von EDF_{dyn}⁺ wäre wie folgt strukturiert: Auf der obersten Ebene kontrolliert ein *Mixed-Criticality-Scheduler* (etwa nach Vestal [28] oder Burns & Davis [29]) die Zuweisung von CPU-Budgets an Anwendungspartitionen. Innerhalb jeder Partition läuft EDF_{dyn}⁺ als lokaler Scheduler und verwaltet die dynamisch ankommenden Tasks der Partition autonom. Neu ankommende Tasks von einem höheren Kritikalitätsniveau können die Partitions Grenzen überschreiten und einen *Cross-Partition-Präemptionsmechanismus* auslösen, der analog zur Ankunftspräemption in Definition 9 formalisiert werden kann. Dieser Ansatz verbindet die bewiesene Optimalität von EDF_{dyn}⁺ auf Partitionsebene mit der Zertifizierbarkeit gemischtkritischer Systeme nach ISO 26262 und DO-178C.

Formale Verifikation von EDF_{dyn}⁺. Die Aussagen des dynamischen Modells – insbesondere Satz 3 und die Lemmata 3 und ?? – sind ebenso wie die statischen Sätze Kandidaten für eine formale Verifikation in UPPAAL, TLA⁺ oder Isabelle/HOL. Die besondere Herausforderung liegt im unendlichen Zustandsraum, der durch beliebig viele Task-Ankünfte entsteht. Ein vielversprechender Ansatz sind *parametrische zeitbehaftete Automaten*, bei denen die Anzahl der Ankünfte A als Parameter behandelt wird und Correctness-Eigenschaften als Funktionen von A nachgewiesen werden können. Alternativ kann eine Abstraktion auf endliche Modelle (bounded model checking) mit wachsenden Schranken $A \leq A_{\text{max}}$ systematisch Gegenbeispiele ausschließen und so das Vertrauen in die Korrektheit des dynamischen Algorithmus sukzessive stärken.

Simulation und Benchmark-Evaluierung. Die in Abschnitt 8 durchgeführten Monte-Carlo-Simulationen betreffen ausschließlich das statische Modell. Analoge Simulationen für EDF_{dyn}⁺ sollten für verschiedene Ankunftsrate λ , Blocking-Wahrscheinlichkeiten p und Systemauslastungen U durchgeführt werden. Besonders interessant sind Szenarien mit kombinierten Ankunfts- und Blocking-Bursts, wie sie in vernetzten Fahrzeugsystemen bei Feldbus-Überlast oder OTA-Update-Kampagnen auftreten können. Ein Benchmark-Vergleich zwischen EDF_{dyn}⁺, dem klassischen dynamischen EDF sowie FIFO und Priority-Inheritance-basiertem RMS würde die theoretischen Optimalitätsaussagen empirisch absichern und die quantitative Überlegenheit in praxisnahen Szenarien demonstrieren.

Literatur

- [1] C.L. Liu und J.W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *Journal of the ACM*, Bd. 20, Nr. 1, S. 46–61, 1973.
- [2] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3. Aufl. New York: Springer, 2011.
- [3] L. Sha, R. Rajkumar und J. P. Lehoczky, “Priority inheritance protocols: An approach to real-time synchronization,” *IEEE Transactions on Computers*, Bd. 39, Nr. 9, S. 1175–1185, 1990.
- [4] W. Hoeffding, “Probability inequalities for sums of bounded random variables,” *Journal of the American Statistical Association*, Bd. 58, Nr. 301, S. 13–30, 1963.
- [5] AUTOSAR Consortium, *AUTOSAR Classic Platform – Specification of Operating System*, Release R23-11, 2023.
- [6] ISO, *ISO 26262: Road Vehicles – Functional Safety*, 2. Aufl., 2018.
- [7] W. Feller, *An Introduction to Probability Theory and Its Applications*, Bd. 1, 3. Aufl. New York: Wiley, 1968.
- [8] T.H. Cormen, C.E. Leiserson, R.L. Rivest und C. Stein, *Introduction to Algorithms*, 4. Aufl. Cambridge, MA: MIT Press, 2022.
- [9] J. A. Stankovic, “Misconceptions about real-time computing: A serious problem for next-generation systems,” *IEEE Computer*, Bd. 21, Nr. 10, S. 10–19, 1988.
- [10] J. P. Lehoczky, L. Sha und Y. Ding, “The rate monotonic scheduling algorithm: Exact characterization and average case behavior,” in *Proc. 10th IEEE Real-Time Systems Symposium (RTSS)*, S. 166–171, 1989.
- [11] M.L. Dertouzos, “Control robotics: The procedural control of physical processes,” in *Proc. IFIP Congress*, S. 807–813, 1974.
- [12] M. Spuri und G. Buttazzo, “Scheduling aperiodic tasks in dynamic priority systems,” *Real-Time Systems*, Bd. 10, Nr. 2, S. 179–210, 1996.
- [13] T. P. Baker, “Stack-based scheduling of realtime processes,” *Real-Time Systems*, Bd. 3, Nr. 1, S. 67–99, 1991.
- [14] T. Abdelzaher, E. G. Coffman Jr. und J. Lehoczky, “Optimal scheduling of push and pull systems,” *ACM SIGMETRICS Performance Evaluation Review*, Bd. 32, Nr. 1, S. 13–24, 2004.
- [15] N. C. Audsley, A. Burns, M. Richardson, K. Tindell und A. J. Wellings, “Applying new scheduling theory to static priority pre-emptive scheduling,” *Software Engineering Journal*, Bd. 8, Nr. 5, S. 284–292, 1993.
- [16] S.K. Baruah, L.E. Rosier und R.R. Howell, “Algorithms and complexity concerning the preemptive scheduling of periodic, real-time tasks on one processor,” *Real-Time Systems*, Bd. 2, Nr. 4, S. 301–324, 1990.

- [17] S. K. Dhall und C. L. Liu, “On a real-time scheduling problem,” *Operations Research*, Bd. 26, Nr. 1, S. 127–140, 1978.
- [18] R. Pellizzoni und G. Lipari, “Improved schedulability analysis of real-time transactions with earliest deadline scheduling,” *ACM Transactions on Embedded Computing Systems*, Bd. 7, Nr. 2, S. 1–19, 2008.
- [19] B. B. Brandenburg und J. H. Anderson, “Optimality results for multiprocessor real-time locking,” in *Proc. 32nd IEEE Real-Time Systems Symposium (RTSS)*, S. 49–60, 2011.
- [20] R. Rajkumar, *Synchronization in Real-Time Systems: A Priority Inheritance Approach*, Dordrecht: Kluwer Academic Publishers, 1991.
- [21] A. K. Mok, *Fundamental Design Problems of Distributed Systems for the Hard-Real-Time Environment*, Diss., Massachusetts Institute of Technology, 1983.
- [22] E. Bini und G. C. Buttazzo, “Schedulability analysis of periodic fixed priority systems,” *IEEE Transactions on Computers*, Bd. 53, Nr. 11, S. 1462–1473, 2004.
- [23] J. H. Anderson, J. M. Calandrino und U. C. Devi, “Real-time scheduling on multicore platforms,” in *Proc. 11th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, S. 179–190, 2005.
- [24] H. Chetto, M. Silly und T. Bouchentouf, “Dynamic scheduling of real-time tasks under precedence constraints,” *Real-Time Systems*, Bd. 2, Nr. 3, S. 181–194, 1990.
- [25] M. G. Harbour, M. H. Klein und J. P. Lehoczky, “Timing analysis for fixed-priority scheduling of hard real-time systems,” *IEEE Transactions on Software Engineering*, Bd. 20, Nr. 1, S. 13–28, 1994.
- [26] G. Lipari und S. K. Baruah, “A hierarchical extension to the earliest deadline first scheduling algorithm,” in *Proc. 6th IEEE Real-Time Technology and Applications Symposium (RTAS)*, S. 123–131, 2000.
- [27] S. Baruah, V. Bonifaci, G. D’Angelo, H. Li, A. Marchetti-Spaccamela, S. van der Ster und L. Stougie, “Preemptive uniprocessor scheduling of mixed-criticality sporadic task systems,” *Journal of the ACM*, Bd. 62, Nr. 2, S. 1–33, 2015.
- [28] S. Vestal, “Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance,” in *Proc. 28th IEEE Real-Time Systems Symposium (RTSS)*, S. 239–243, 2007.
- [29] A. Burns und R. Davis, *Mixed Criticality Systems – A Review*, v5.0, Univ. of York, Department of Computer Science, 2013.